



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

PROGRAMA DE POSGRADOS

PROYECTO FINAL

INFRAESTRUCTURA DE RED SEGURA

Autor (es):

Quezada Gallardo, Angelo Diancarlo – aquezadag1@miumg.edu.gt

Sosa Arroyo, Danny Josué – dsosaa1@miumg.edu.gt

Rodríguez Díaz, Carlos Alejandro – crodriguez10@miumg.edu.gt

Profesor

MOREIRA FLORES, LUIS MARIANO

Curso

SEGURIDAD DE REDES TCP/IP

Guatemala, 2026

Nota: 100/100

ÍNDICE

INTRODUCCIÓN.....	i
MARCO TEÓRICO	1
Metodología y Enfoque de Trabajo	3
Enfoque conceptual.....	3
Plataforma y entorno de trabajo.....	3
Fases de ejecución	3
Herramientas y criterios de validación	4
Forma de trabajo del equipo	5
Hallazgos	5
Hallazgos derivados del reconocimiento y análisis de superficie de ataque	6
Hallazgos del análisis de vulnerabilidades web.....	6
Hallazgos de detección y monitoreo	7
Riesgos identificados	7
Fortalezas y Debilidades.....	8
Fortalezas	8
Debilidades y limitaciones.....	9
Oportunidades de mejora	9
Mejores Prácticas Implementadas	10
Segmentación de red y defensa en profundidad	10
Principios de Zero Trust en identidad y acceso	10
Autenticación multifactor reforzada	10
Monitoreo, correlación y visibilidad en capas.....	10
Endurecimiento de servicios críticos.....	11
Evaluación de seguridad con metodología controlada	11
Documentación y reproducibilidad	11
Lecciones Aprendidas	11
Integración de herramientas open source en entornos simulados	11
La detección no sustituye al endurecimiento preventivo	12
Importancia de la coherencia entre diseño y controles de identidad	12

Diferencias en madurez de controles entre servicios	12
Valor de las pruebas controladas para la toma de decisiones	12
Documentación como componente de seguridad	12
DISEÑO Y PLANIFICACIÓN	13
TOPOLOGÍA DE RED	13
DIRECCIONAMIENTO IP	14
INTEGRACIÓN Y DESPLIEGUE DE METODOLOGÍAS Y CONCEPTOS EN SEGURIDAD DE REDES	15
FIREWALL PERIMETRAL – PFSENSE	15
VPN – OPENVPN	15
DETECCIÓN DE INTRUSIONES – SURICATA	16
ANÁLISIS DE TRÁFICO DE RED – ZEEK	17
SERVIDOR WEB – NGINX	17
SERVIDOR DNS – BIND9	19
GESTIÓN DE IDENTIDAD – KEYCLOAK	19
SIEM – WAZUH.....	20
CONTROLES DE SEGURIDAD APLICADOS	21
SEGMENTACIÓN DE RED POR ZONAS DE SEGURIDAD	21
CONTROL DE ACCESO – REGLAS ACL	21
DETECCIÓN Y CORRELACIÓN DE EVENTOS – WAZUH.....	22
AUTENTICACIÓN MULTIFACTOR – KEYCLOAK MFA.....	23
ACCESO REMOTO SEGURO – OPENVPN	25
MONITOREO CENTRALIZADO DE AGENTES	25
PRUEBA Y VALIDACIONES.....	26
METODOLOGÍA DE PRUEBAS.....	26
RECONOCIMIENTO – NMAP	26
RECONOCIMIENTO CONTRA NGINX-DMZ	27
RECONOCIMIENTO CONTRA BLIND09-DNS.....	28
RECONOCIMIENTO CONTRA KEYCLOAK.....	28
ANÁLISIS DE VULNERABILIDADES WEB – NIKTO.....	29
FUERZA BRUTA – HYDRA.....	31

FUERZA BRUTA SSH CONTRA NGINX-DMZ	31
FUERZA BRUTA HTTP CONTRA KEYCLOAK.....	31
ENUMERACIÓN DNS – ZONE TRANSFER ATTACK	32
VALIDACIÓN DE DETECCIÓN – WAZUH SIEM.....	34
CONCLUSIONES.....	37
RECOMENDACIONES	38
REFERENCIAS BIBLIOGRAFICAS	40

TABLAS

Tabla 1 Hallazgos.....	5
Tabla 2 Direcciones de consola VNC	14
Tabla 3 Hallazgos identificados.....	30
Tabla 4 Detección Wazuh.	34

FIGURAS

Figura 1	13
Figura 2 Nodos listados.....	14
Figura 3 pfSense Dashboard.....	15
Figura 4 Configuración de OpenVPN en pfSense.....	16
Figura 5 Suricata Activo en pfSense sobre la interfaz WAN.....	16
Figura 6 Zeek activo en pfSense sobre la interfaz con la red local 10.0.0.0/8.....	17
Figura 7 Estado del servicio Nginx activo en Nginx-DMZ.	18
Figura 8 Estado del servicio Bind9	19
Figura 9 Panel de administración de Keycloak accesible en http://10.0.30.11:8080	20
Figura 10 Dashboard de Wazuh mostrando los 4 agentes registrados y activos.	20
Figura 11 Reglas de Firewall en pfSense.	21
Figura 12 Reglas ACL configuradas	22
Figura 13 Archivo local_rules.xml con las 3 reglas de detección personalizadas configuradas en Wazuh.....	22
Figura 14 Configuración de Required Actions en Keycloak con Configure OTP activado como acción predeterminada.	23

Figura 15 Pantalla Mobile Authenticator Setup solicitando configuración del segundo factor al usuario.	24
Figura 16 Acceso exitoso al portal de cuenta de Keycloak.	25
Figura 17 Lista de agentes Wazuh registrados mostrando los 4 nodos monitoreados con sus IPs y estado.	26
Figura 18 Puestos abiertos y versiones de servicios expuestas.	27
Figura 19 Versión de BIND expuesta.	28
Figura 20 Keycloak mostrando el puerto 8080 abierto sin TLS.	29
Figura 21	30
Figura 22 Hydra ejecutando ataque de fuerza bruta SSH contra Nginx-DMZ con intentos visibles.	31
Figura 23 Hydra bloqueado por Keycloak tras 4 intentos.	32
Figura 24 Resultado de dig AXFR mostrando Transfer failed contra BIND9-DNS.	33
Figura 25 Resultado de dnsrecon confirmando que la transferencia de zona fue denegada.	33
Figura 26 Dashboard de Security Events en Wazuh mostrando 793 alertas con el pico de ataque visible en el gráfico.	34
Figura 27 Lista de eventos de seguridad mostrando las alertas de fuerza bruta con rule ID 5763.	35
Figura 28 Detalle de la rule 5763 mostrando frequency 8, timeframe 120 y mapeo MITRE ATT&CK.	35
Figura 29 Dashboard filtrado por rule.id 5763 mostrando 2 alertas de Brute Force detectadas.	36

INTRODUCCIÓN

En el panorama actual de la ciberseguridad, donde las amenazas evolucionan constante y los perímetros de red tradicionales se diluyen, las organizaciones se enfrentan al desafío de proteger su infraestructura crítica sin sacrificar la operatividad. El presente proyecto, desarrollado en el marco del curso Seguridad de Redes TCP/IP, aborda esta problemática mediante el diseño, implementación y validación de una infraestructura de red segmentada y segura, emulando un entorno empresarial de pequeña escala.

El objetivo fundamental de este trabajo no se limitó al mero despliegue de herramientas de seguridad, sino que buscó demostrar la comprensión práctica de los controles implementados, sus límites operativos y su efectividad frente a vectores de ataque realistas. Para ello, se construyó una topología integral en GNS3, utilizando exclusivamente soluciones de código abierto, donde se integraron componentes como un firewall perimetral (pfSense), un sistema de detección de intrusiones (Suricata), un analizador de tráfico de red (Zeek), un servidor web (Nginx), un servidor DNS (BIND9), un gestor de identidades (Keycloak) y un SIEM centralizado (Wazuh).

El valor añadido de este proyecto reside en su enfoque holístico y práctico. Más allá de la configuración inicial, se sometió a la infraestructura a una batería de pruebas ofensivas controladas desde una estación Kali Linux, simulando el comportamiento de un atacante. Este ejercicio permitió identificar hallazgos críticos —como versiones de software desactualizadas, ausencia de cifrado TLS o exposición innecesaria de servicios—, validar la eficacia de controles como la autenticación multifactor (MFA) y el bloqueo de transferencias de zona DNS, y demostrar la capacidad de detección y correlación del SIEM Wazuh. El resultado es un análisis crítico y documentado que transita desde la teoría hasta la evidencia tangible, ofreciendo lecciones

MARCO TEÓRICO

Para la construcción de la infraestructura de red segura, se integraron diversas tecnologías de código abierto, cada una con un rol específico dentro de la estrategia de defensa en profundidad. A continuación, se describen los fundamentos y propósitos de cada una.

1. **GNS3 (Graphical Network Simulator-3):** Es una plataforma de simulación y emulación de redes que permite diseñar y desplegar topologías de red complejas sin necesidad de hardware físico. Su flexibilidad para integrar dispositivos virtuales, sistemas operativos reales y aplicaciones de seguridad lo convirtió en el entorno central para este proyecto, facilitando pruebas controladas y reproducibles.
2. **pfSense:** Se trata de un firewall y enrutador de código abierto basado en FreeBSD, ampliamente utilizado en entornos empresariales. En este proyecto, pfSense actuó como núcleo de la topología, gestionando la segmentación de red en seis zonas de seguridad (WAN, DMZ, LAN de usuarios, LAN de servidores, administración y WiFi), aplicando políticas de control de acceso (ACL) basadas en el principio de mínimo privilegio, y albergando los servicios de VPN y detección de intrusiones.
3. **Suricata:** Es un motor de detección y prevención de intrusiones (IDS/IPS) de alto rendimiento. A diferencia de otros sistemas, Suricata puede utilizar múltiples hilos de procesamiento para analizar el tráfico en tiempo real. En este proyecto, se desplegó en la interfaz WAN de pfSense para inspeccionar el tráfico entrante y saliente, generando eventos en formato EVE JSON que pueden ser correlacionados por un SIEM.
4. **Zeek (antiguamente Bro):** Es un analizador de tráfico de red (NDR) que se enfoca en generar metadatos de alto nivel a partir del tráfico que observa. A diferencia de un IDS/IPS, Zeek no busca firmas de ataques de forma nativa, sino que extrae información detallada sobre sesiones (TCP/UDP/ICMP), consultas DNS, transacciones HTTP, certificados TLS, etc. Esto lo convierte en una herramienta invaluable para labores de caza de amenazas (threat hunting) y análisis forense.
5. **Wazuh:** Es una plataforma de gestión de seguridad y monitorización (SIEM y XDR) de código abierto. Se compone de un servidor central que analiza los datos recibidos de unos agentes instalados en los equipos a monitorizar. Wazuh permite la correlación de eventos de seguridad a nivel de host (logs, integridad de archivos, detección de rootkits)

y de red, alertando sobre comportamientos maliciosos y generando una trazabilidad completa de los incidentes.

6. **Keycloak:** Es una solución de gestión de identidades y accesos (IAM) de código abierto, especializada en la autenticación y autorización de aplicaciones y servicios modernos. Keycloak centraliza la gestión de usuarios, soporta estándares como OAuth 2.0, OpenID Connect y SAML, y permite implementar controles de seguridad robustos como la Autenticación Multifactor (MFA) mediante TOTP (Time-based One-Time Password), como se hizo en este proyecto.
7. **Nginx:** Es un servidor web, proxy inverso y balanceador de carga de alto rendimiento. Su arquitectura basada en eventos lo hace extremadamente eficiente en la gestión de conexiones concurrentes. En este proyecto, Nginx se utilizó como servidor web en la zona DMZ para exponer un servicio público simulado, evaluando posteriormente su nivel de hardening y configuración de seguridad.
8. **BIND9 (Berkeley Internet Name Domain):** Es la implementación de referencia del protocolo DNS (Domain Name System). Se utiliza para traducir nombres de dominio legibles por humanos (ej. proyecto.local) a direcciones IP. En este proyecto, BIND9 se configuró como servidor DNS interno, proporcionando resolución de nombres para todos los dispositivos de la infraestructura. Un control crítico aplicado fue la restricción de las transferencias de zona (AXFR), evitando que un atacante pudiera enumerar todos los registros internos.
9. **OpenVPN:** Es una solución de código abierto para crear redes privadas virtuales (VPN) robustas. Utiliza una combinación de técnicas criptográficas estándar de la industria para establecer un canal de comunicación seguro y cifrado a través de redes no confiables. En este proyecto, se configuró OpenVPN sobre pfSense, empleando certificados digitales emitidos por una autoridad certificadora interna para garantizar la autenticación mutua y el acceso administrativo remoto seguro a la red.
10. **Herramientas de Prueba (Kali Linux):** Kali Linux es una distribución especializada en pruebas de penetración y seguridad informática. De ella se utilizaron herramientas como **Nmap** (escaneo de puertos y detección de servicios), **Nikto** (escáner de vulnerabilidades web), **Hydra** (ataque de fuerza bruta a servicios de red)

y **dig/dnsrecon** (enumeración y consulta DNS), todas ejecutadas de forma controlada para validar la postura de seguridad de la infraestructura.

Metodología y Enfoque de Trabajo

El desarrollo del proyecto siguió una metodología secuencial con los laboratorios prácticos del curso Seguridad de Redes TCP/IP adoptando un enfoque orientado a demostrar no solo el despliegue de herramientas, sino la comprensión de los controles implementados, sus límites operativos y su efectividad frente a vectores de ataque realistas.

Enfoque conceptual

La estrategia de seguridad se fundamentó en construir un entorno empresarial pequeño basado en dos principios complementarios, defensa en profundidad y Zero Trust. La primera se materializó mediante la segmentación de la red en zonas de confianza, la detección de intrusiones, el análisis de tráfico y la centralización de eventos. La segunda se aplicó principalmente en identidad y acceso con Keycloak mediante autenticación multifactor obligatoria y acceso administrativo remoto cifrado mediante OpenVPN con certificados emitidos por una autoridad certificadora interna.

Plataforma y entorno de trabajo

La infraestructura se desplegó íntegramente en GNS3, utilizada como plataforma principal de integración, simulación y emulación. Sobre esta base se incorporaron dispositivos virtuales, sistemas operativos y herramientas open source para conseguir un entorno apto para pruebas controladas sin afectar sistemas productivos. El núcleo de la topología lo constituyó pfSense configurado con seis interfaces de red correspondientes a las zonas WAN, DMZ, LAN de usuarios, LAN de servidores, red de administración y red inalámbrica.

Fases de ejecución

La ejecución del proyecto se organizó en cuatro fases interdependientes:

1. **Planificación y diseño.** En esta etapa se definió la topología lógica, el direccionamiento IP por zona, la ubicación de los servicios críticos y la matriz de flujos permitidos. Se establecieron, entre otras, las redes DMZ, LAN de usuarios, servidores, administración, LAN WiFi y túnel VPN. Los servicios se distribuyeron de forma coherente con el nivel de exposición de cada zona: Nginx en DMZ, Wazuh, Keycloak y BIND9 en LAN de

servidores, y Kali Linux en la red de administración como estación de pruebas ofensivas.

2. Implementación e integración.

Sobre la topología base se desplegaron los componentes: firewall perimetral (pfSense), IDS/IPS (Suricata sobre WAN), NDR (Zeek sobre LAN), servidor web (Nginx), DNS interno (BIND9), gestión de identidad (Keycloak), SIEM (Wazuh) y VPN administrativa (OpenVPN). Durante esta fase se documentaron configuraciones, estados de servicios y evidencias de funcionamiento mediante capturas de pantalla.

3. Endurecimiento y aplicación de controles.

Se implementaron controles de segmentación, reglas ACL inter-VLAN con enfoque de mínimo privilegio, reglas personalizadas de detección en Wazuh, MFA en Keycloak mediante y monitoreo centralizado mediante agentes Wazuh en los nodos críticos. Cabe señalar que, por incompatibilidad de FreeIPA con la versión de Ubuntu Server empleada, se optó por Keycloak como solución de identidad, decisión técnica que fue documentada en la implementación.

4. Pruebas, validación y análisis.

Las pruebas ofensivas se ejecutaron de forma controlada desde Kali Linux, simulando el comportamiento de un atacante interno o externo. Se emplearon herramientas de reconocimiento y explotación (Nmap 7.98, Nikto 2.6.0, Hydra 9.6, dig y dnsrecon), complementadas con la validación de detección en Wazuh. Este ciclo permitió mostrar la postura de seguridad antes y después de las pruebas, generar evidencia técnica y alimentar el análisis de hallazgos.

Herramientas y criterios de validación

La selección de herramientas respondió a la capa de seguridad que cada componente debía cubrir. Suricata proporcionó detección y prevención de intrusiones con eventos en formato EVE JSON, Zeek generó metadatos de red para hunting y análisis forense, Wazuh consolidó logs, correlacionó eventos y generó alertas, Keycloak centralizó autenticación y MFA, y OpenVPN aseguró el acceso administrativo remoto.

Las pruebas se diseñaron para cubrir reconocimiento, análisis de vulnerabilidades, fuerza bruta controlada, enumeración DNS y verificación de detección en SIEM. La validación no se limitó a confirmar que los servicios estuvieran activos, sino que buscó demostrar

trazabilidad de eventos, alertas y capacidad de respuesta ante escenarios de ataque simulados.

Forma de trabajo del equipo

La documentación del informe final se distribuyó entre los integrantes según la estructura acordada, marco teórico e introducción, diseño e implementación con evidencias, secciones analíticas de metodología, hallazgos, lecciones aprendidas y cierre. Esta distribución permitió mantener coherencia entre la implementación técnica, las pruebas ejecutadas y el análisis crítico del resultado obtenido.

Hallazgos

Los hallazgos presentados a continuación derivan del cruce entre las pruebas controladas, el análisis de vulnerabilidades, la observación del comportamiento de los servicios y la correlación de eventos en Wazuh. La Tabla 4 consolida los ocho hallazgos más relevantes identificados durante la evaluación.

Tabla 1

Hallazgos técnicos identificados durante las pruebas de validación

#	Hallazgo	Severidad	Remediación propuesta
1	nginx/1.18.0 desactualizado	Media	Actualizar a versión 1.24 o superior
2	Headers HTTP de seguridad ausentes	Media	Configurar en nginx.conf
3	Versión de BIND expuesta	Media	Establecer version "none" en named.conf
4	Keycloak sin TLS	Alta	Configurar certificado SSL
5	SSH expuesto en DMZ	Media	Restringir acceso por dirección IP
6	Transferencia de zona DNS bloqueada	Correcto	Mantener configuración actual
7	Hydra bloqueado por rate-limit en Keycloak	Correcto	Mantener configuración actual
8	MFA operativo en Keycloak	Correcto	Mantener configuración actual

--	--	--	--

Hallazgos derivados del reconocimiento y análisis de superficie de ataque

Durante la fase de reconocimiento con Nmap, para simular un escenario en el que ICMP está bloqueado, se confirmó la exposición de servicios en los objetivos evaluados. Contra Nginx-DMZ se identificaron los puertos 22 de OpenSSH y 80 de Nginx, con la versión exacta del servidor web visible en la respuesta HTTP. El puerto 443 apareció cerrado, lo que evidenció la ausencia de HTTPS en el servicio expuesto en DMZ. Este comportamiento incrementa la superficie de ataque al facilitar la identificación de vulnerabilidades conocidas asociadas a versiones específicas de software.

Contra **BIND9-DNS (10.0.30.12)** se detectaron los puertos 22 y 53, identificando ISC BIND. Un aspecto relevante fue la exposición de la versión exacta mediante el registro bind.version, información que un atacante podría utilizar para localizar CVEs aplicables.

Contra **Keycloak (10.0.30.11)** se identificó el puerto 8080 con el panel de administración accesible sobre HTTP, sin cifrado TLS. Los puertos 80 y 443 permanecieron cerrados, confirmando que las credenciales de autenticación podrían transmitirse en texto plano si no se implementa HTTPS.

Hallazgos del análisis de vulnerabilidades web

El escaneo con Nikto contra el servidor Nginx en DMZ generó varios miles de solicitudes HTTP en 35 segundos e identificó ocho hallazgos de seguridad relacionados con configuración y endurecimiento. Entre ellos destacan: versión desactualizada de Nginx (1.18.0 frente a 1.24.0 como referencia), ausencia de headers Content-Security-Policy, Referrer-Policy, X-Content-Type-Options, Strict-Transport-Security y Permissions-Policy, así como el uso de X-Frame-Options en configuración considerada obsoleta. Estos resultados confirman que, aunque el servicio web es funcional, su configuración no cumple con buenas prácticas actuales de hardening HTTP.

Hallazgos de pruebas ofensivas controladas

En la prueba de fuerza bruta SSH contra Nginx-DMZ, Hydra 9.6 ejecutó intentos paralelos utilizando el diccionario RockYou (14.344.399 contraseñas), simulando un escenario realista de ataque por credenciales débiles. Los intentos fueron visibles en la

ejecución de la herramienta, lo que permitió posteriormente validar su detección en el SIEM.

En contraste, el ataque de fuerza bruta HTTP contra Keycloak fue bloqueado tras únicamente cuatro intentos, mediante el mecanismo integrado de protección contra fuerza bruta, que devolvió el mensaje "all children were disabled due to many connection errors". Este comportamiento demuestra una diferencia significativa en la capacidad de resistencia entre servicios expuestos sin controles adicionales (SSH en DMZ) y un sistema de identidad con rate-limiting activo.

Las pruebas de enumeración DNS mediante dig AXFR y dnsrecon confirmaron que la transferencia de zona contra BIND9 falló de forma consistente (Transfer failed; No records were returned), indicando que el control de zone transfer está correctamente restringido y que un atacante no puede obtener un mapa completo de registros internos por este vector.

Hallazgos de detección y monitoreo

Tras la ejecución de las pruebas ofensivas, Wazuh registró un total de 793 alertas durante el periodo de evaluación, con un pico visible en el dashboard de Security Events. Se documentaron 151 eventos de autenticación fallida asociados al ataque de Hydra contra SSH. La regla personalizada 5763 (fuerza bruta SSH, nivel 10, frequency 8, timeframe 120) generó alertas correlacionadas con el marco MITRE ATT&CK de Brute Force, confirmando la trazabilidad del ataque desde la estación Kali hacia el objetivo en DMZ. También se activaron las reglas 2502, 5758 y 5760, asociadas a múltiples intentos fallidos, exceso de intentos y autenticación fallida, respectivamente.

Estos resultados evidencian que la infraestructura no solo implementa controles preventivos, sino que dispone de capacidad de detección y correlación que permite identificar patrones de ataque en tiempo casi real.

Riesgos identificados

Con base en la evidencia recopilada, se identificaron los siguientes riesgos prioritarios:

- **Riesgo alto:** transmisión de credenciales sin cifrado en Keycloak (puerto 8080/HTTP), susceptible a interceptación en escenarios de acceso no confiable.
- **Riesgo medio:** exposición de SSH en un servidor ubicado en DMZ, ampliando la superficie de ataque hacia un servicio de administración remota.

- **Riesgo medio:** divulgación de versiones de software (Nginx, BIND) que facilita la planificación de explotación dirigida.
- **Riesgo medio:** configuración web incompleta en headers de seguridad, incrementando la probabilidad de ataques del lado cliente (clickjacking, MIME sniffing, entre otros).

Los controles correctamente implementados bloqueo de zone transfer, rate-limiting en Keycloak y MFA operativo mitigan riesgos específicos, pero no compensan por sí solos las debilidades estructurales identificadas en cifrado y hardening.

Fortalezas y Debilidades

Fortalezas

La principal fortaleza del proyecto reside en la arquitectura segmentada y coherente con un modelo de seguridad empresarial. La implementación de seis zonas de confianza gestionadas por pfSense, con reglas ACL de mínimo privilegio, reduce de forma significativa el movimiento lateral y establece una base sólida para controles adicionales.

En identidad y acceso, el despliegue de Keycloak con MFA obligatorio mediante TOTP constituye un control robusto. La configuración de OTP como acción requerida para todos los usuarios del realm impide el acceso sin segundo factor, combinando algo que se sabe (contraseña), algo que se tiene (dispositivo autenticador) y un código temporal de validez limitada (30 segundos). Las pruebas confirmaron que el mecanismo funciona correctamente, incluyendo la pantalla de configuración del autenticador móvil y el acceso exitoso tras completar el segundo factor.

La capacidad de detección centralizada representa otra fortaleza relevante. Wazuh, con cuatro agentes activos (Nginx-DMZ, Keycloak, Kali-Linux y el servidor central), demostró visibilidad operativa mediante la generación de 793 alertas y la activación de reglas personalizadas frente a fuerza bruta SSH. La integración de Suricata (IDS/IPS), Zeek (NDR) y Wazuh (SIEM) conforma un esquema de visibilidad en capas alineado con las recomendaciones del curso.

Asimismo, se verificaron controles defensivos específicos bien configurados, la denegación de transferencias de zona DNS y el bloqueo automático de Hydra en Keycloak evidencian que ciertos vectores clásicos de ataque fueron neutralizados de forma efectiva.

El acceso administrativo remoto mediante OpenVPN con certificados emitidos por la CA interna ProyectoFinal-CA refuerza la autenticación mutua y el cifrado del canal, complementando la segmentación física/lógica de la red de administración.

Debilidades y limitaciones

A pesar de las fortalezas descritas, el análisis identificó limitaciones técnicas que deben reconocerse de forma objetiva.

La debilidad más crítica corresponde a Keycloak operando sin TLS, con el panel accesible en HTTP (puerto 8080). En un entorno productivo, esta condición expone credenciales y tokens a interceptación, invalidando parcialmente el valor del MFA en escenarios de red no confiable.

La presencia de SSH expuesto en un host de DMZ constituye una debilidad de diseño. Aunque el servicio puede ser necesario para administración, su exposición en una zona orientada a servicios públicos incrementa el riesgo de compromiso inicial, especialmente cuando se combina con ataques de fuerza bruta, como se demostró con Hydra.

En el plano de endurecimiento de servicios, Nginx presenta una versión desactualizada (1.18.0) y carece de headers HTTP de seguridad recomendados, según lo reportado por Nikto. BIND9, por su parte, expone su versión exacta, facilitando reconocimiento dirigido.

Desde la perspectiva del ciclo de evaluación, el proyecto demostró detección efectiva de ataques SSH, pero la documentación disponible no evidencia una remediación comparativa completa (antes/después) para todos los hallazgos identificados. Esta limitación documental no invalida los resultados obtenidos, pero sí señala un área de mejora para la trazabilidad del proceso de endurecimiento.

Finalmente, el segmento inalámbrico (10.0.50.0/24) forma parte del diseño topológico, pero la documentación práctica revisada no incluye pruebas específicas de seguridad WiFi ni evidencias de evaluación criptográfica de ese segmento, lo que constituye una brecha entre el diseño planificado y la validación ejecutada.

Oportunidades de mejora

Las debilidades identificadas abren líneas claras de mejora: implementación de HTTPS en Keycloak y Nginx, restricción de SSH en DMZ por IP o reubicación del servicio

en zona de administración, actualización de componentes, configuración de headers de seguridad, ocultamiento de versiones en BIND y ejecución de un ciclo formal de remediación con nueva evaluación comparativa.

Mejores Prácticas Implementadas

Durante la ejecución del proyecto se aplicaron prácticas técnicas, operativas y de seguridad alineadas con estándares profesionales.

Segmentación de red y defensa en profundidad

Se implementó segmentación estricta en seis zonas de seguridad (WAN, DMZ, LAN usuarios, LAN servidores, administración y WiFi), gestionadas centralmente por pfSense. Cada zona representa un nivel de confianza diferenciado y el tráfico inter-zona se controla mediante reglas de firewall explícitas, aplicando el principio de denegación por defecto y permitiendo únicamente los flujos necesarios para la operación de los servicios. Esta práctica reduce el movimiento lateral y constituye la base del modelo de defensa en profundidad adoptado.

Principios de Zero Trust en identidad y acceso

Aunque la infraestructura mantiene zonas de confianza tradicionales, se incorporaron elementos centrales de Zero Trust en identidad: autenticación centralizada con Keycloak, políticas de acceso por realm, MFA obligatorio y acceso administrativo remoto autenticado mediante certificados digitales en OpenVPN. Estas medidas refuerzan la verificación continua de identidad antes de conceder acceso a recursos sensibles.

Autenticación multifactor reforzada

Keycloak se configuró con OTP (TOTP, RFC 6238) como acción requerida para todos los usuarios, compatible con aplicaciones autenticadoras estándar (Google Authenticator, FreeOTP, Microsoft Authenticator). Esta implementación satisface el requisito de MFA reforzada para accesos sensibles y fue validada mediante prueba de login exitoso con usuario de prueba tras configuración del segundo factor.

Monitoreo, correlación y visibilidad en capas

Se desplegó un esquema de visibilidad distribuida: Suricata para IDS/IPS en perímetro (WAN), Zeek para análisis de tráfico y metadatos de red (LAN), y Wazuh como SIEM centralizado con agentes en nodos críticos. Se definieron al menos tres casos de uso de detección en local_rules.xml: fuerza bruta SSH, escaneo de puertos y modificación de

archivos críticos del sistema. Esta arquitectura permite correlacionar eventos de red, host e identidad, trascendiendo la dependencia exclusiva de reglas de firewall.

Endurecimiento de servicios críticos

Se aplicaron controles de hardening en múltiples componentes: restricción de transferencias de zona en BIND9, rate-limiting integrado en Keycloak contra fuerza bruta, publicación controlada del servicio web en DMZ y centralización de logs para trazabilidad. OpenVPN utiliza una PKI interna (ProyectoFinal-CA) para autenticación mutua, práctica recomendada para acceso administrativo remoto.

Evaluación de seguridad con metodología controlada

Las pruebas ofensivas se ejecutaron desde un entorno de laboratorio aislado (Kali Linux en red de administración), empleando herramientas reconocidas en la industria (Nmap, Nikto, Hydra, dig, dnsrecon) y validando los resultados en Wazuh. Esta práctica de validación activa permite contrastar la efectividad de los controles que han sido implementados.

Documentación y reproducibilidad

La arquitectura se documentó con topología GNS3, tabla de direccionamiento IP, capturas de configuración y evidencias de pruebas, respondiendo al requisito de reproducibilidad. La trazabilidad entre diseño, implementación, prueba y hallazgo constituye una práctica operativa esencial para la operación y mejora continua de entornos de ciberseguridad.

Lecciones Aprendidas

La ejecución de este proyecto permitió al equipo consolidar aprendizajes técnicos y metodológicos relevantes para futuras implementaciones de infraestructura segura.

Integración de herramientas open source en entornos simulados

Uno de los aprendizajes centrales fue la viabilidad de construir un SOC en miniatura funcional utilizando exclusivamente herramientas open source sobre GNS3. La integración de pfSense, Suricata, Zeek, Wazuh, Keycloak, BIND9 y Nginx demostró que es posible reproducir un entorno empresarial con controles de seguridad modernos sin depender de soluciones propietarias, siempre que exista planificación previa, documentación rigurosa y validación activa de cada componente.

La detección no sustituye al endurecimiento preventivo

Las pruebas evidenciaron que Wazuh detectó de forma efectiva el ataque de fuerza bruta SSH (793 alertas, regla 5763 activa), pero el servicio SSH en DMZ permaneció expuesto durante toda la prueba. Esto ilustra una lección fundamental: la visibilidad y la alerta temprana son indispensables, pero no reemplazan la reducción de superficie de ataque. Un enfoque maduro de ciberseguridad debe combinar prevención, detección, respuesta y remediación.

Importancia de la coherencia entre diseño y controles de identidad

La sustitución de FreeIPA por Keycloak, motivada por incompatibilidades con Ubuntu Server, subraya la necesidad de validar compatibilidad de plataformas en fases tempranas del diseño. Aunque Keycloak resultó una alternativa adecuada con MFA funcional, la decisión tardía podría haber generado retrabajo en un contexto con plazos más estrictos.

Diferencias en madurez de controles entre servicios

El contraste entre el bloqueo inmediato de Hydra en Keycloak (cuatro intentos) y la persistencia del ataque SSH contra Nginx reveló que no todos los servicios incorporan controles de protección con el mismo nivel de madurez. Keycloak incluye rate-limiting nativo; Nginx y OpenSSH requieren configuración adicional (fail2ban, restricciones en pfSense, deshabilitación de autenticación por contraseña). Esta observación refuerza la necesidad de evaluar cada servicio expuesto de forma individual, sin asumir que la segmentación de red compensa debilidades a nivel de aplicación.

Valor de las pruebas controladas para la toma de decisiones

La ejecución secuencial de reconocimiento (Nmap), análisis de vulnerabilidades (Nikto), explotación controlada (Hydra), enumeración DNS (dig/dnsrecon) y validación en SIEM (Wazuh) proporcionó un ciclo completo de evaluación que transformó configuraciones teóricas en evidencia medible. Las pruebas no solo confirmaron vulnerabilidades, sino también fortalezas (zone transfer bloqueado, MFA operativo, detección de fuerza bruta), lo que enriquece el análisis y evita conclusiones unilaterales.

Documentación como componente de seguridad

El proyecto reforzó que la documentación técnica topologías, tablas de direccionamiento, reglas de firewall, reglas de detección, capturas de pruebas y tablas de

hallazgos no es un complemento administrativo, sino un activo operativo que permite reproducir, auditar y mejorar la infraestructura. La trazabilidad entre evidencia y conclusión es especialmente valorada en contextos académicos y profesionales de ciberseguridad.

DISEÑO Y PLANIFICACIÓN

TOPOLOGÍA DE RED

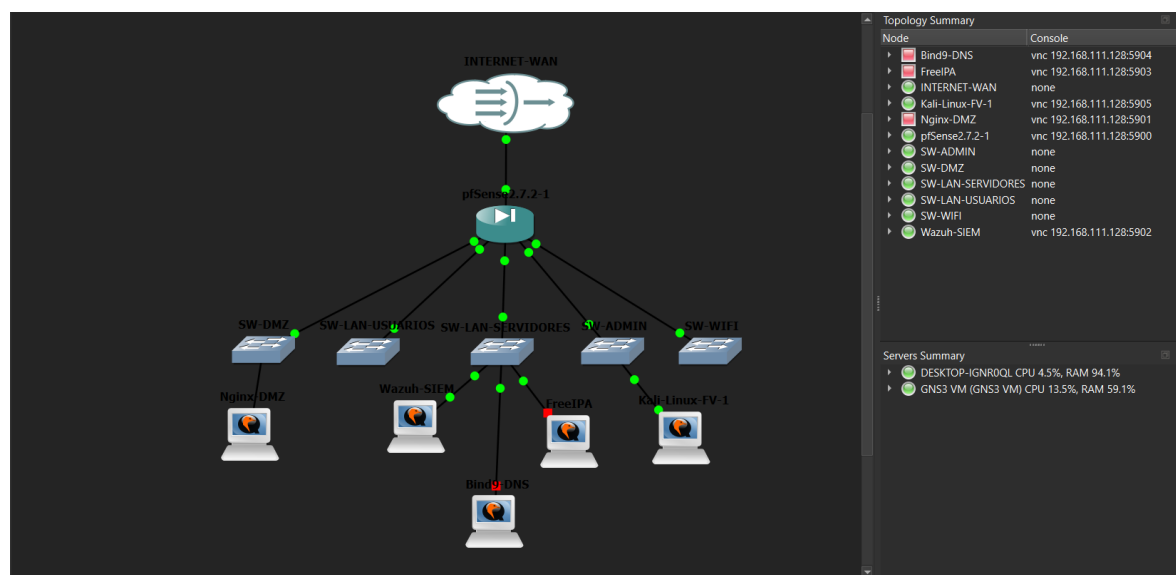
Se implementó una topología de red empresarial segmentada por zona de seguridad, siguiendo el principio de defensa en profundidad y los fundamentos de Zero Trust, por lo que la infraestructura fue desplegada íntegramente en GNS3, agregando dispositivos virtuales, sistemas operativos y herramientas open source.

Pfsense siendo el núcleo de la topología, es el firewall que actúa como punto central de control y segmentación, gestionando seis interfaces de red que corresponden a las siguientes zonas:

- WAN – Conexión hacia internet simulado.
- DMZ (10.0.10.0/24) – Zona desmilitarizada para servicios públicos.
- LAN Usuarios (10.0.20.0/24) – Red de estaciones de trabajo.
- LAN Servidores (10.0.30.0/24) – Red de servidores internos críticos.
- Red Admin (10.0.40.0/24) – Red de administración y gestión.
- Red WiFi (10.0.50.0/25) – Segmento inalámbrico aislado.

Figura 1

Topología GNS3



DIRECCIONAMIENTO IP

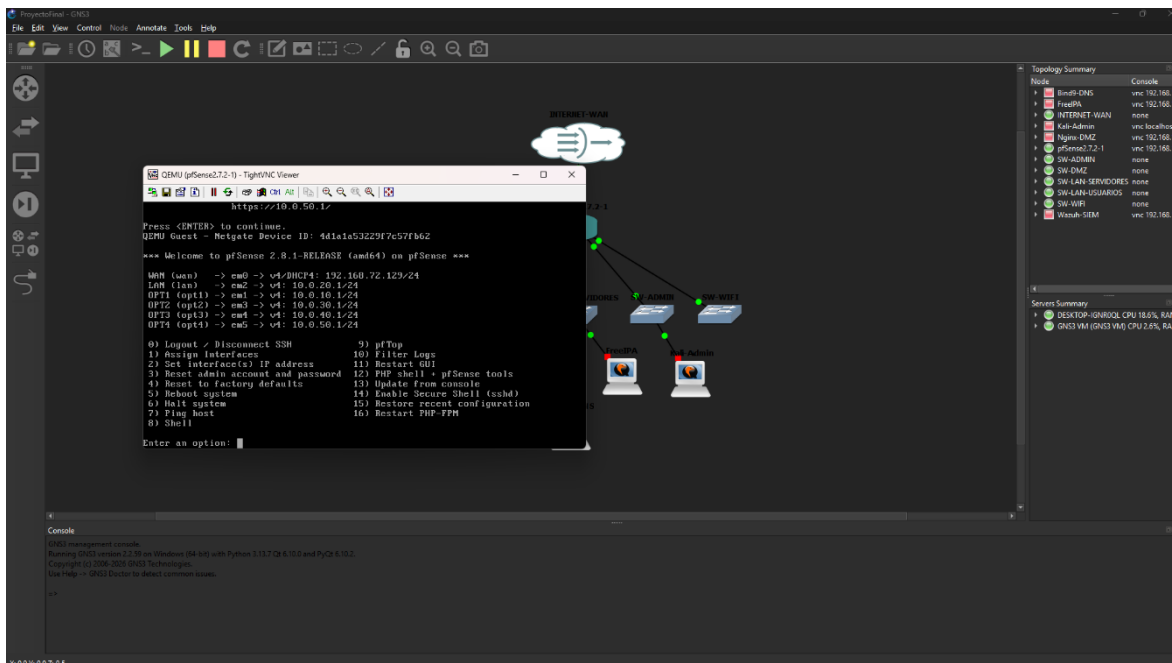
Tabla 2

Direcciones de consola VNC

Zona	Red	Gateway	Dispositivos
WAN	DHCP	ISP	pfSense em0
DMZ	10.0.10.0/24	10.0.10.1	Nginx-DMZ (10.0.10.10)
LAN Usuarios	10.0.20.0/24	10.0.20.1	Estaciones de trabajo
LAN Servidores	10.0.30.0/24	10.0.30.1	Wazuh (10.0.30.10), Keycloak (10.0.30.11), Blind9 (10.0.30.12)
Admin	10.0.40.0/24	10.0.40.1	Kali-Linux (10.0.40.10)
WiFi	10.0.50.0/24	10.0.50.1	Dispositivos inalámbricos
VPN	10.0.100.0/24	-	Túnel Open VPN

Figura 2

Nodos listados.

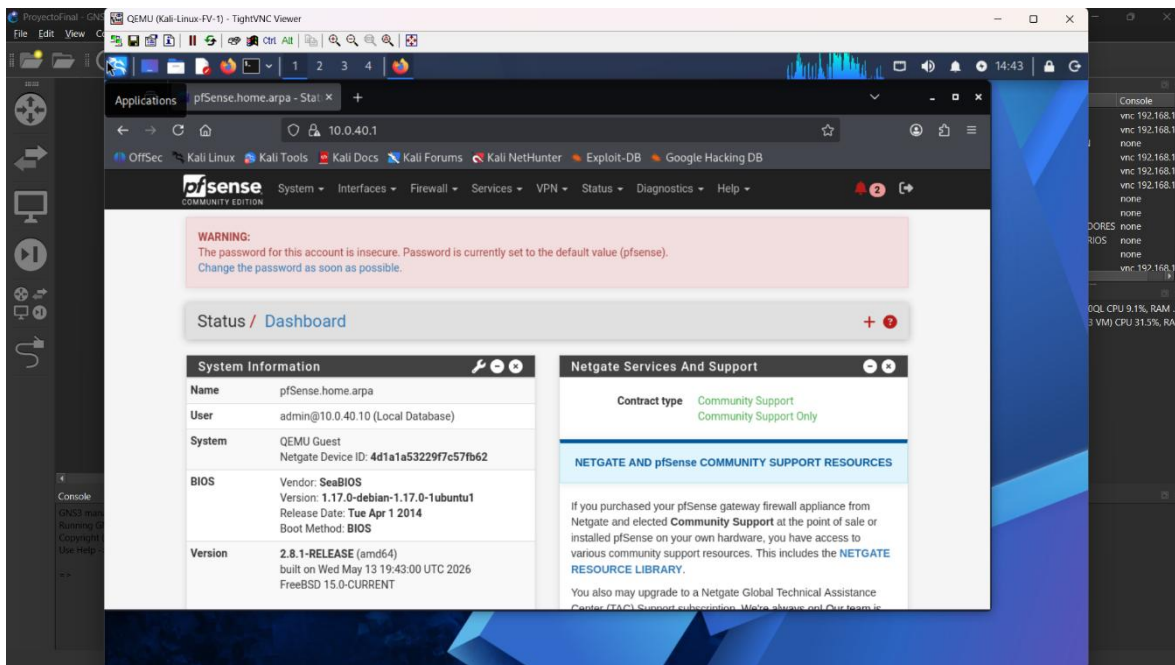


INTEGRACIÓN Y DESPLIEGUE DE METODOLOGÍAS Y CONCEPTOS EN SEGURIDAD DE REDES

FIREWALL PERIMETRAL – PFSENSE

Se desplegó pfSense Community Edition 2.7.2 siendo el Firewall de toda la topología, configurado con seis interfaces de red, cada una correspondiente a una zona de seguridad definida en el diseño, por lo que su función principal es controlar y filtrar el tráfico entre zonas, aplicar reglas de acceso inter-VLAN, gestionar el NAT hacia la WAN y centralizar los servidores de VPN.

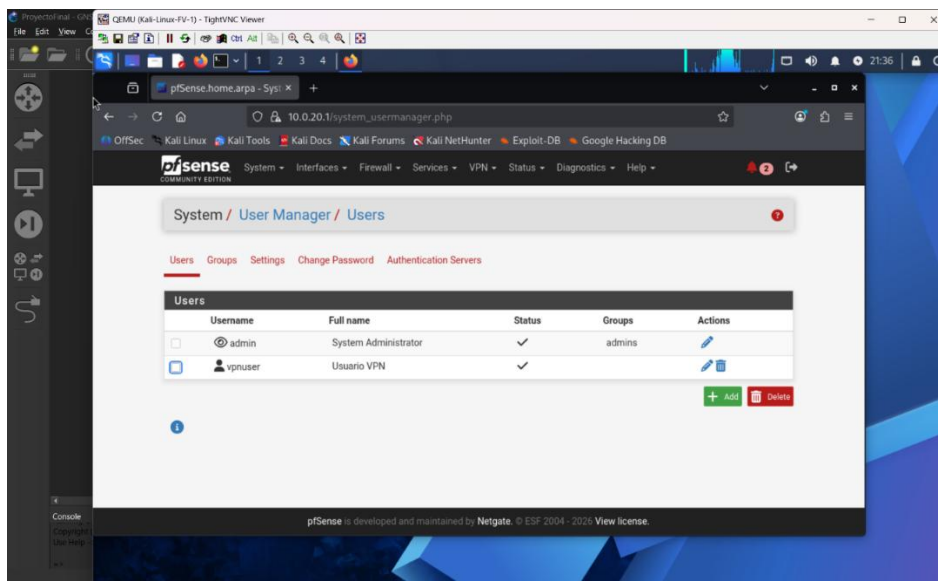
Figura 3
pfSense Dashboard.



VPN – OPENVPN

OpenVPN se configuró de forma directa en pfSense para proporcionar acceso administrativo remoto seguro, se creó una Autoridad Certificadora propia denominada ProyectoFinal-CA, un usuario VPN con credenciales robustas y un túnel dedicado en la red 10.0.100.0/24, utilizando un mecanismo que garantiza el acceso administrativo a la infraestructura la cual se puede realizar a través de un canal cifrado.

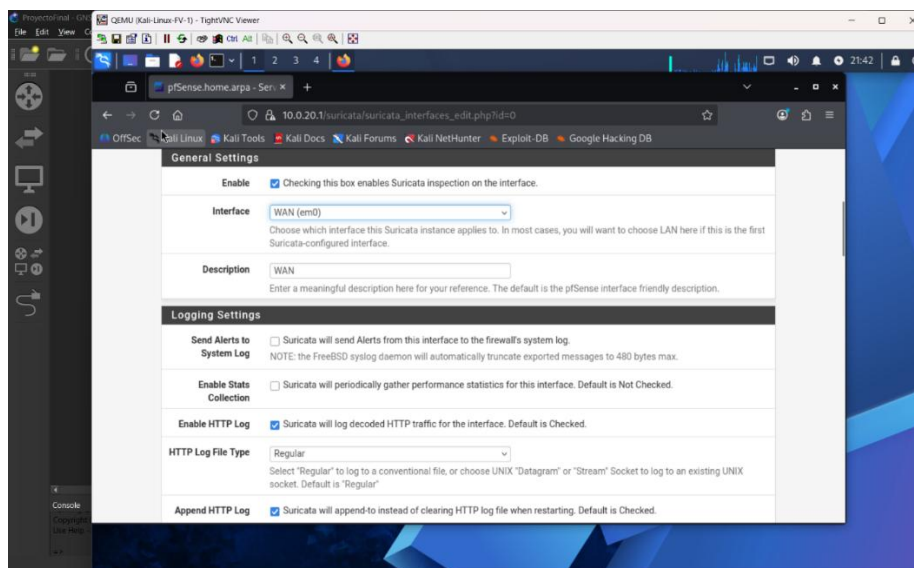
Figura 4
Configuración de OpenVPN en pfSense.



DETECCIÓN DE INTRUSIONES – SURICATA

Se realizó la instalación de Suricata en pfSense sobre la interfaz WAN como parte del sistema de detección y prevención de intrusiones, teniendo en cuenta que este analiza el tráfico en tiempo real generando eventos en formato EVE JSON, los cuales pueden ser integrados con el SIEM para correlación de alertas.

Figura 5
Suricata Activo en pfSense sobre la interfaz WAN.

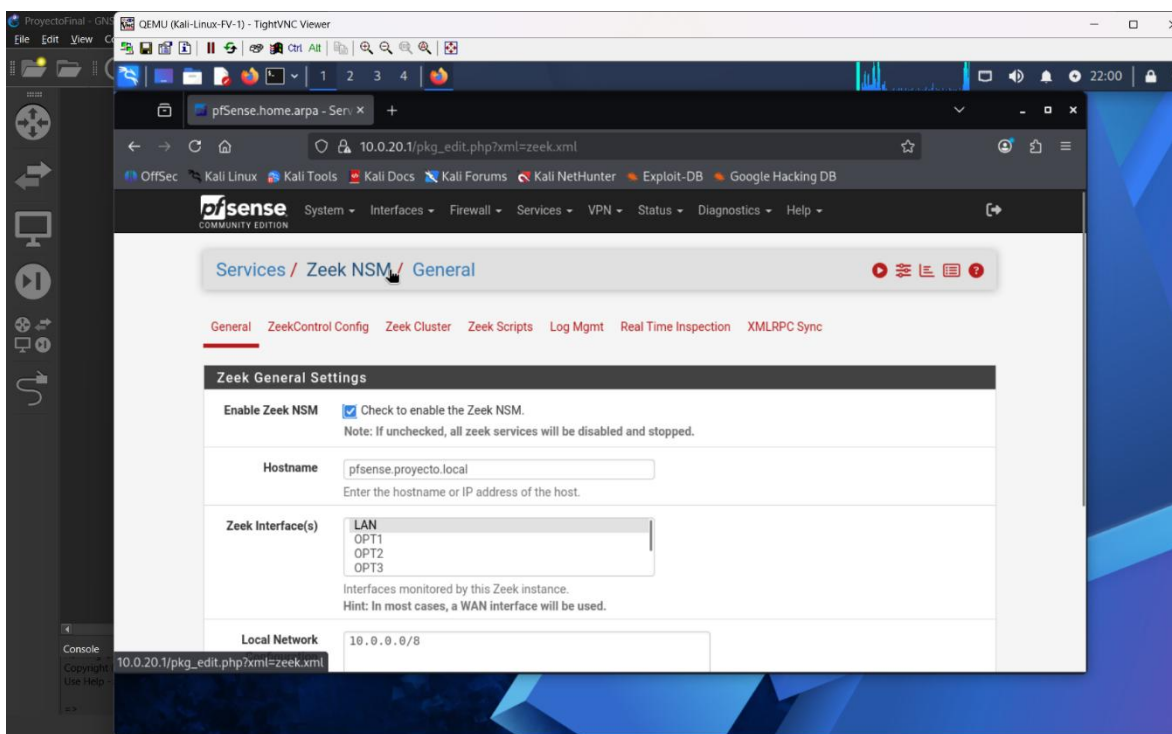


ANÁLISIS DE TRÁFICO DE RED – ZEEK

Se desplegó Zeek en pfSense sobre la interfaz LAN como componente NDR, lo cual genera metadatos de red detallados incluyendo sesiones TCP/UDP, consultas DNS, transacciones HTTP y handshakes TLS, proporcionando visibilidad profunda de tráfico interno para actividades de hunting y análisis forense.

Figura 6

Zeek activo en pfSense sobre la interfaz con la red local 10.0.0.0/8.

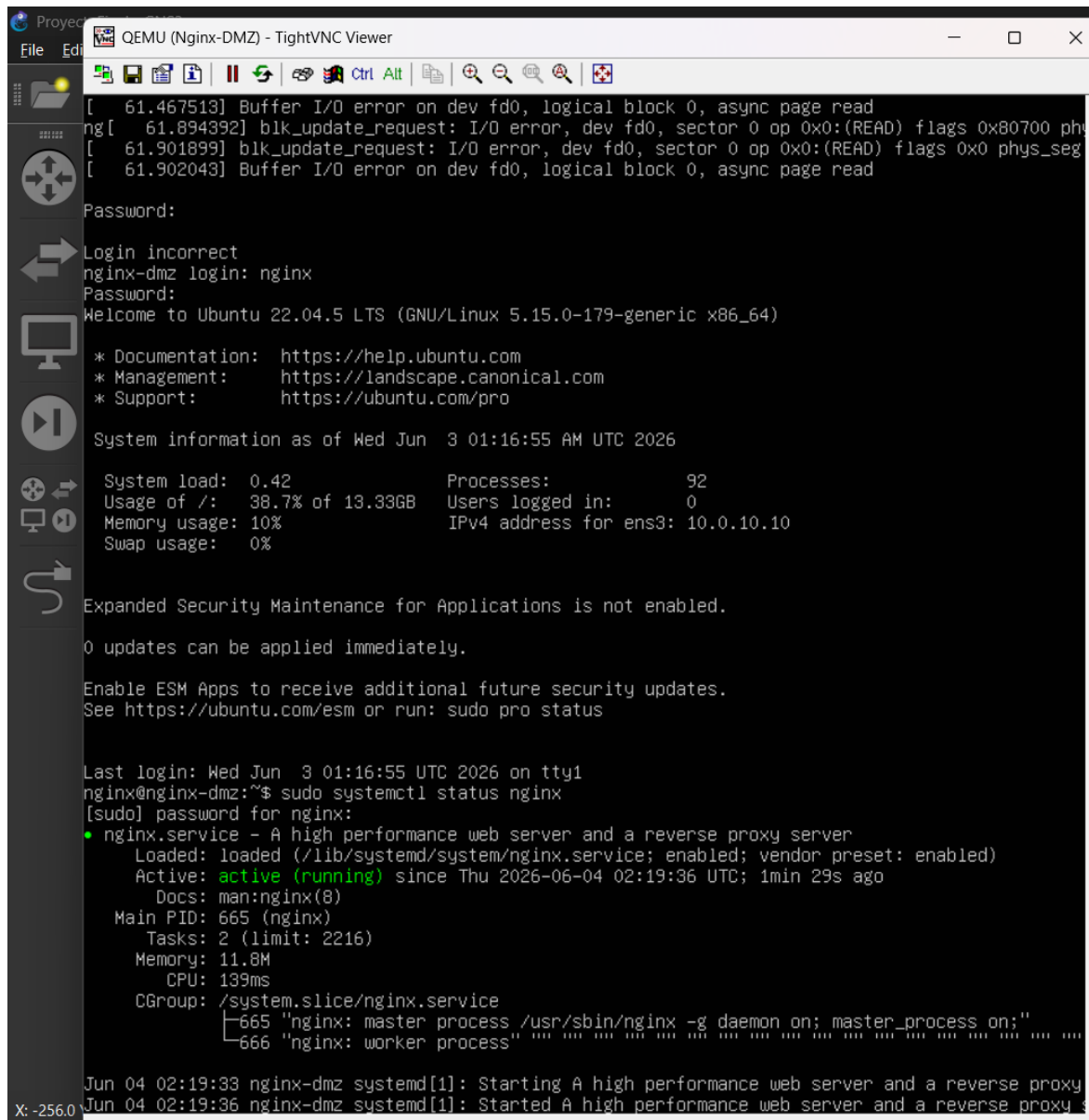


SERVIDOR WEB – NGINX

Nginx 1.18.0 se instaló en la zona DMZ 10.0.10.10 como servidor web que simula un servicio publicado de forma controlada hacia internet, esta página web del proyecto fue configurada y verificada como funcional, representando la superficie de ataque expuesta que fue posteriormente evaluada durante las pruebas de seguridad.

Figura 7

Estado del servicio Nginx activo en Nginx-DMZ.



```
QEMU (Nginx-DMZ) - TightVNC Viewer
[ 61.467513] Buffer I/O error on dev fd0, logical block 0, async page read
[ 61.894392] blk_update_request: I/O error, dev fd0, sector 0 op 0x0:(READ) flags 0x80700 phys_seg
[ 61.901899] blk_update_request: I/O error, dev fd0, sector 0 op 0x0:(READ) flags 0x0 phys_seg
[ 61.902043] Buffer I/O error on dev fd0, logical block 0, async page read
Password:
Login incorrect
nginx-dmz login: nginx
Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-179-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jun  3 01:16:55 AM UTC 2026

System load:  0.42           Processes:           92
Usage of /:   38.7% of 13.33GB Users logged in:       0
Memory usage: 10%           IPv4 address for ens3: 10.0.10.10
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jun  3 01:16:55 UTC 2026 on tty1
nginx@nginx-dmz:~$ sudo systemctl status nginx
[sudo] password for nginx:
• nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-06-04 02:19:36 UTC; 1min 29s ago
     Docs: man:nginx(8)
   Main PID: 665 (nginx)
    Tasks: 2 (limit: 2216)
   Memory: 11.8M
      CPU: 139ms
   CGroup: /system.slice/nginx.service
           └─665 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─666 "nginx: worker process"

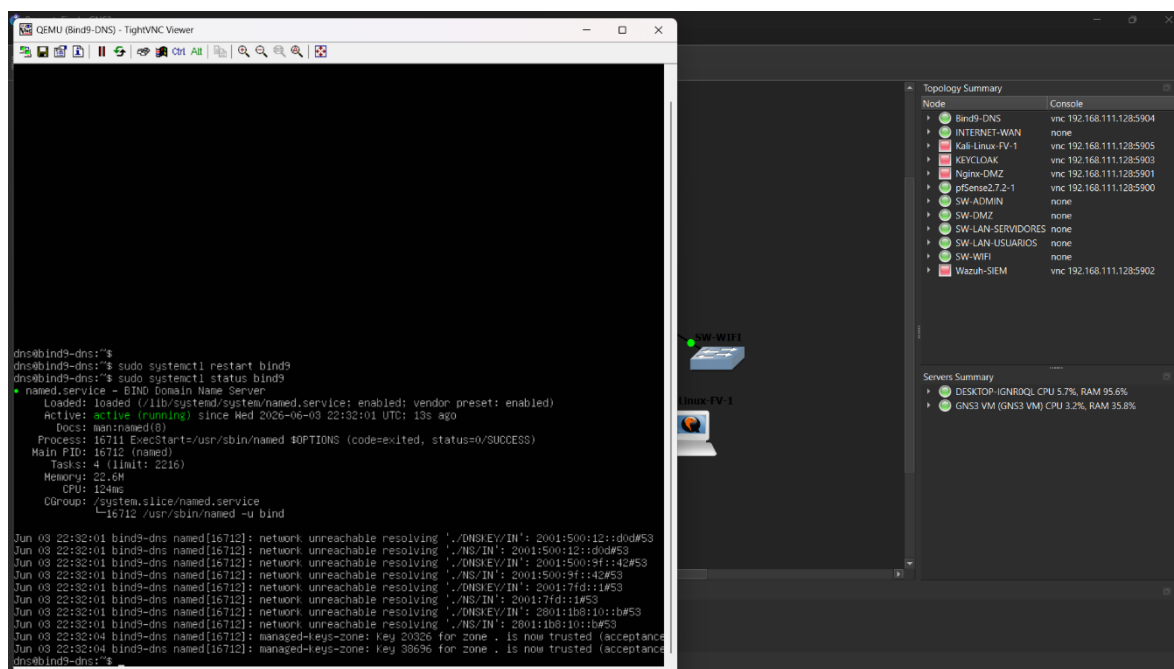
Jun 04 02:19:33 nginx-dmz systemd[1]: Starting A high performance web server and a reverse proxy
Jun 04 02:19:36 nginx-dmz systemd[1]: Started A high performance web server and a reverse proxy
```

SERVIDOR DNS – BIND9

Se configuró BIND9 9.18.39 en la LAN de servidores 10.0.30.12 como servidor DNS interno, configurando la zona directa proyecto.local y la zona inversa correspondiente, dando resolución de nombre para todos los dispositivos de la infraestructura interna.

Figura 8

Estado del servicio Bind9

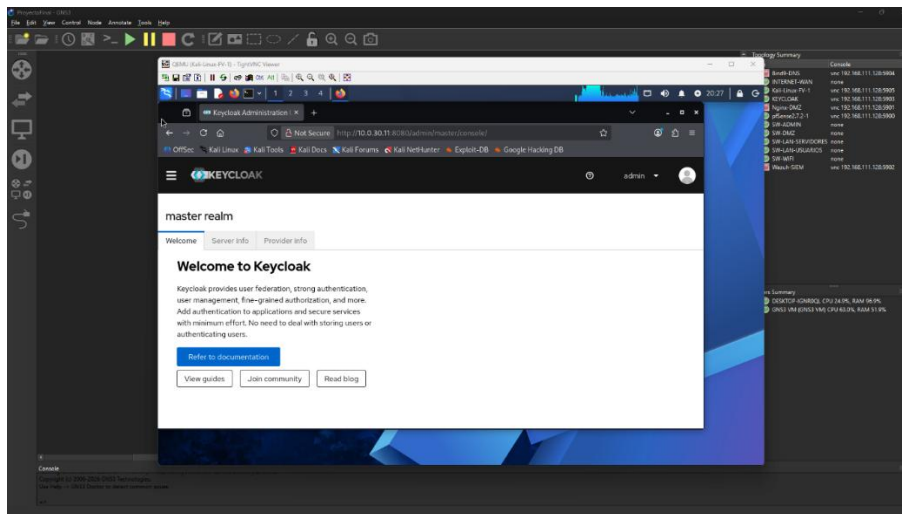


GESTIÓN DE IDENTIDAD – KEYCLOAK

Se utilizó Keycloak en vez de FreeIPA ya que las versiones de este mismo son incompatibles con Ubuntu-Server, centralizamos la utenticación de usuarios, el cual gestiona políticas de acceso y proporciona soporte configurado y verificado como accesible en el puerto 8080.

Figura 9

Panel de administración de Keycloak accesible en <http://10.0.30.11:8080>.

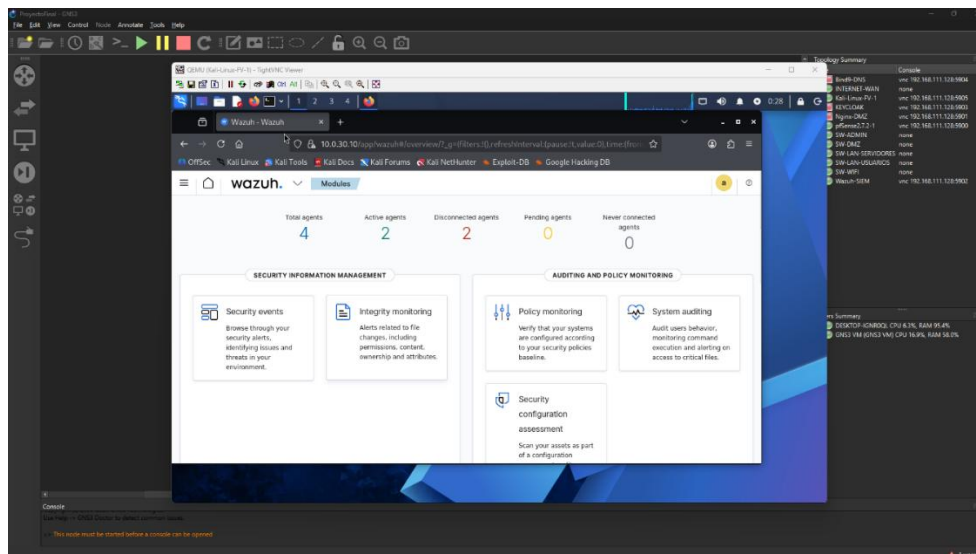


SIEM – WAZUH

Se desplegó Wazuh 4.7.5 en la LAN de servidores 10.0.30.10 como plataforma centralizada de gestión de eventos de seguridad o mas conocida como SIEM, Wazuh recolecta logs de todos los agentes instalados en la infraestructura, correlaciona eventos, lo cual genera alertas y proporciona visibilidad completa del estado de seguridad del entorno, en este se instalaron agentes Wazuh versión 4.7.5 en los nodos Nginx-DMZ, Keycloak y Kali-Linux, todos registrados y activos en el servidor central.

Figura 10

Dashboard de Wazuh mostrando los 4 agentes registrados y activos.



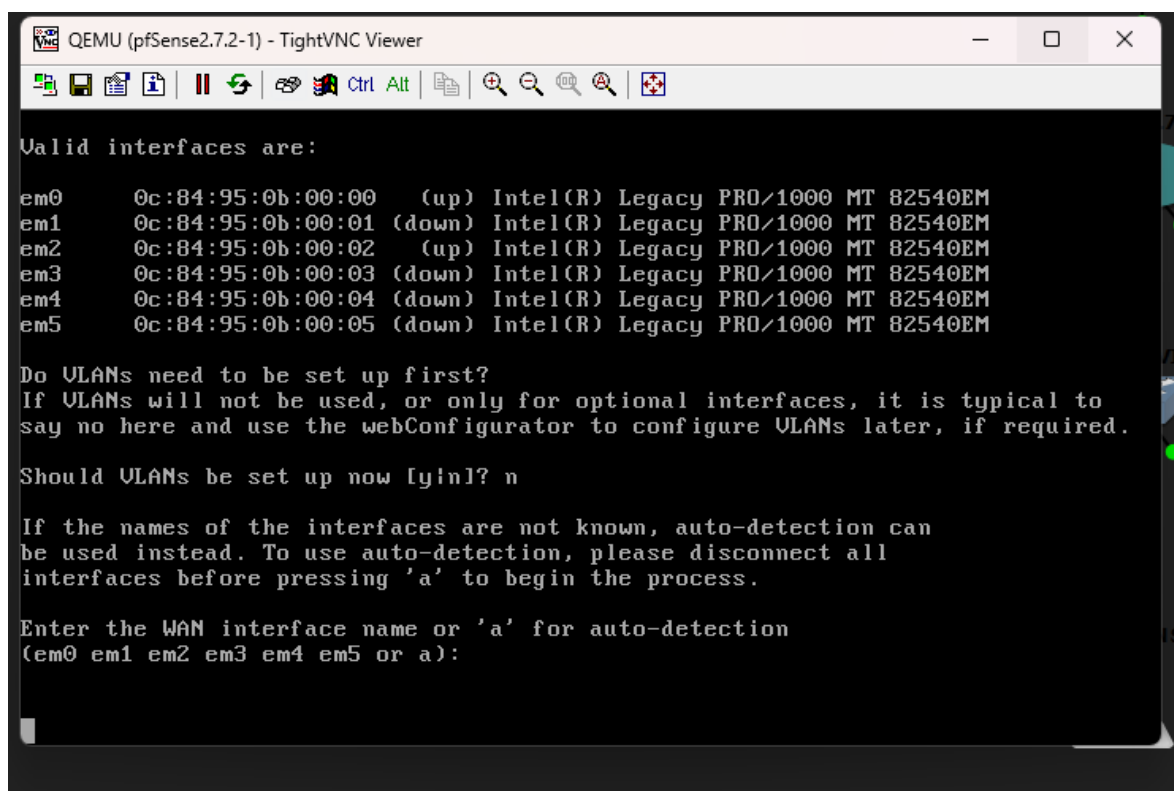
CONTROLES DE SEGURIDAD APLICADOS

SEGMENTACIÓN DE RED POR ZONAS DE SEGURIDAD

Uno de los controles importantes implementados es la segmentación estricta de la red en seis zonas de seguridad claramente delimitadas, gestionadas por pfSense. Cada zona representa un nivel de confianza diferente y el tráfico entre ellas está controlado por reglas de firewall especificadas, lo cual reduce significativamente el riesgo de movimiento lateral en caso de que un atacante comprometa un segmento de la red, ya que no puede acceder libremente a otras zonas sin pasar por los controles de firewall.

Figura 11

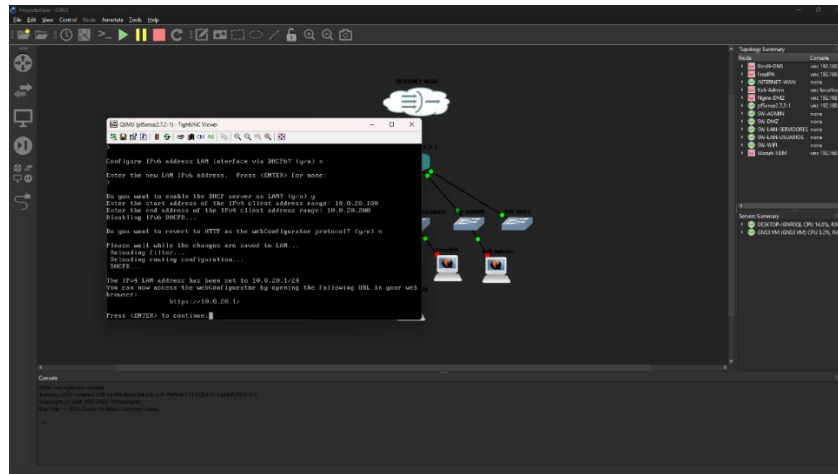
Reglas de Firewall en pfSense.



CONTROL DE ACCESO – REGLAS ACL

Se configuran listas de control de acceso en pfSense para cada zona de seguridad, definiendo explícitamente qué tráfico está permitido y denegando todo lo demás por defecto, las reglas fueron diseñadas siguiendo el principio de mínimo privilegio, permitiendo únicamente los flujos de tráfico necesarios para la operación de cada servicio.

Figura 12
Reglas ACL configuradas



DETECCIÓN Y CORRELACIÓN DE EVENTOS – WAZUH

Esta sección parte de los casos de uso de detección y correlación de eventos en Wazuh mediante reglas personalizadas en el archivo `local_rules.xml`:

- Caso de uso 1: Detección de fuerza bruta SSH.
- Caso de uso 2: Detección de escaneo de puertos.
- Caso de uso 3: Modificación de archivos críticos del sistema.

Teniendo estos tres casos de uso podremos identificar cualquier alteración o ataque desde nuestro servidor.

Figura 13
Archivo `local_rules.xml` con las 3 reglas de detección personalizadas configuradas en Wazuh.

```

GNU nano 6.2 /var/ossec/etc/rules/local_rules.xml *
<!-- Reglas personalizadas - Proyecto Final Seguridad de Redes TCP/IP 2026 -->

<!-- CASO 1: Detección de fuerza bruta SSH -->
<group name="local,syslog,sshd,">
  <rule id="100001" level="10" frequency="5" timeframe="120">
    <if_matched_sid>5716</if_matched_sid>
    <description>Posible ataque de fuerza bruta SSH: multiples intentos fallidos</description>
    <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>

<!-- CASO 2: Deteccion de escaneo de puertos -->
<group name="local,syslog,firewall,">
  <rule id="100002" level="8" frequency="10" timeframe="60">
    <if_matched_sid>4100</if_matched_sid>
    <description>Posible escaneo de puertos detectado desde misma IP</description>
    <group>recon,pci_dss_11.4,</group>
  </rule>
</group>

<!-- CASO 3: Modificacion de archivos criticos del sistema -->
<group name="local,syscheck,">
  <rule id="100003" level="12">
    <if_sid>550</if_sid>
    <match>/etc/passwd|/etc/shadow|/etc/sudoers|/etc/ssh/ssh_config</match>
    <description>Modificacion de archivo critico del sistema detectado</description>
    <group>integrity_check,pci_dss_11.5,</group>
  </rule>
</group>
  
```

AUTENTICACIÓN MULTIFACTOR – KEYCLOAK MFA

Se implementa la autenticación multifactor o MFA en Keycloak como control de seguridad para el acceso a servidores críticos, lo cual el mecanismo utilizado es TOTP basado en el estándar RFC 6238, compatible con aplicaciones autenticadoras como Google Authenticator, FreeOTP y Microsoft Authenticator.

Esta configuración establece que el OTP es una acción requerida para todos los usuarios del real, lo que significa que ningún usuario puede acceder sin haber configurado previamente su segundo factor de autenticación, lo cual implementa efectivamente un esquema de autenticación de múltiples factores que combina:

- Factor 1 – Algo que sabes: Contraseña del usuario.
- Factor 2 – Algo que tienes: dispositivo móvil con app autenticadora
- Factor 3 – Código temporal: OTP de 6 dígitos válido por 30 segundos.

Figura 14

Configuración de Required Actions en Keycloak con Configure OTP activado como acción predeterminada.

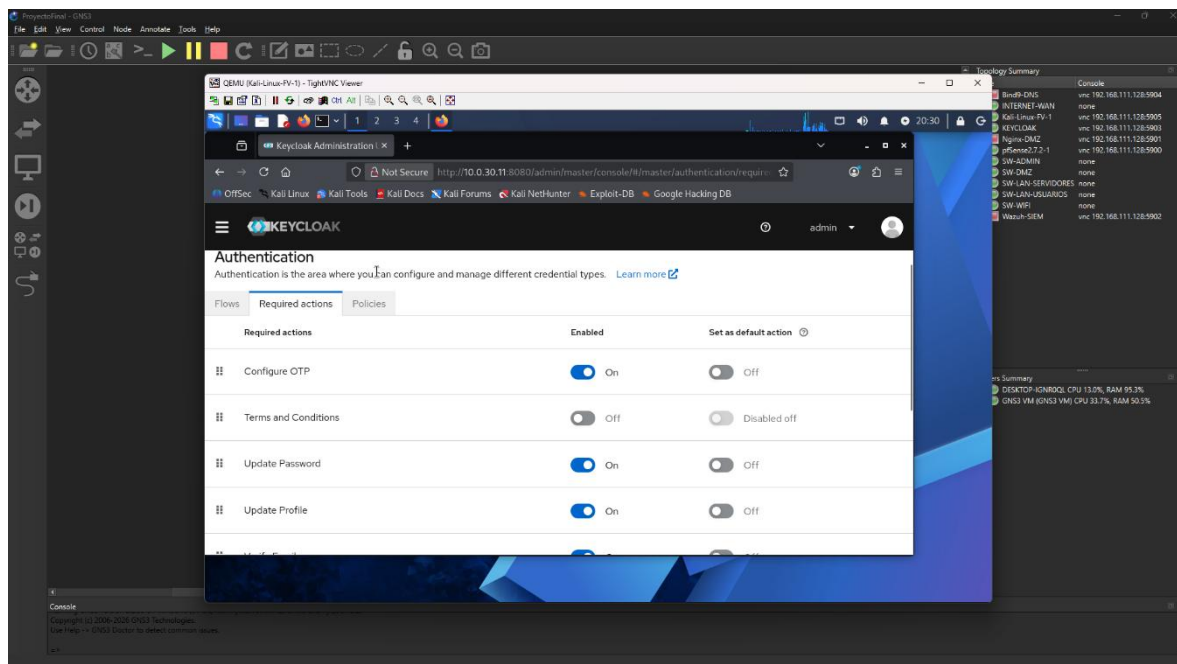


Figura 15

Pantalla Mobile Authenticator Setup solicitando configuración del segundo factor al usuario.

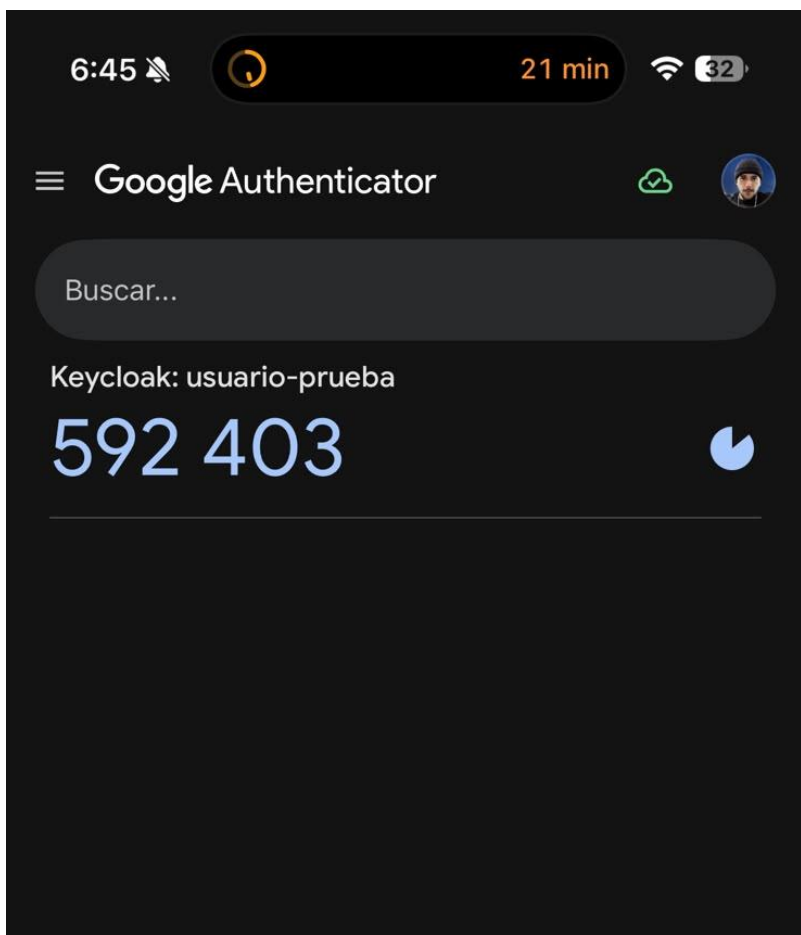
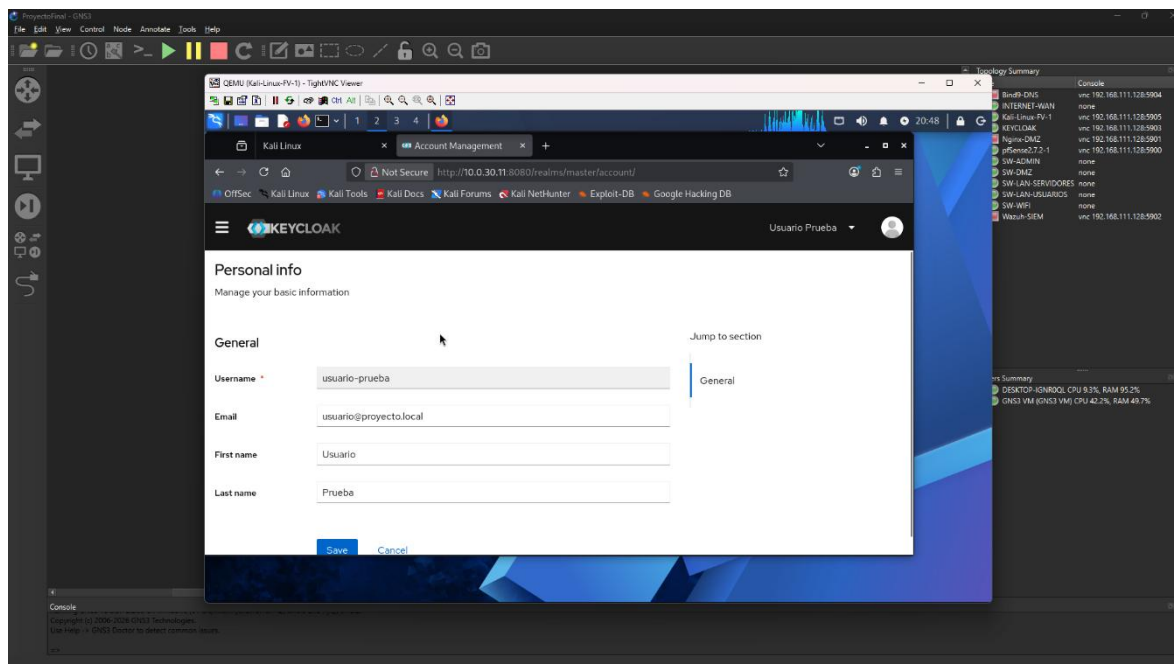


Figura 16

Acceso exitoso al portal de cuenta de Keycloak.



ACCESO REMOTO SEGURO – OPENVPN

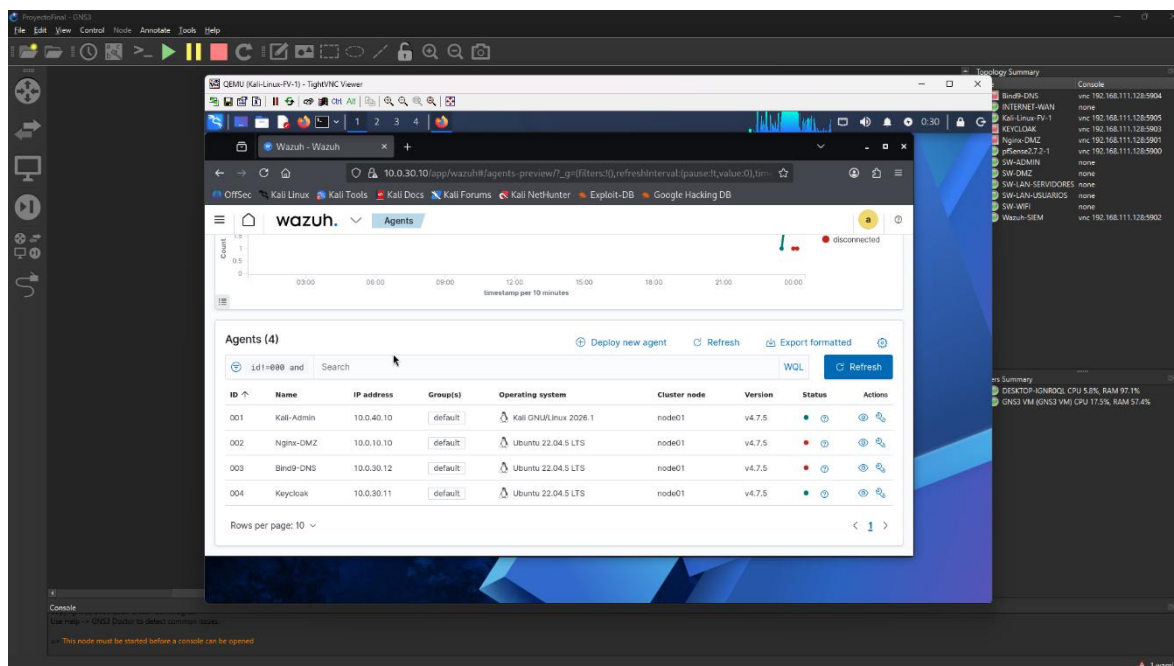
Se implementó OpenVPN como mecanismo de acceso administrativo remoto seguro, utiliza certificados digitales emitidos por la Autoridad Certificadora ProyectoFinal-CA para autenticar tanto el servidor como los clientes, garantizando que únicamente usuarios autorizados con certificado válido puedan establecer conexión con la red de administración.

MONITOREO CENTRALIZADO DE AGENTES

En Wazuh se instalaron agentes en cada servidor crítico de la infraestructura, proporcionando visibilidad completa de eventos de seguridad en tiempo real, a lo que cada agente se encarga de recolectar logs del sistema operativo, eventos de autenticación, cambios del sistema de archivos y actividad de procesos, enviándoles al servidor Wazuh central para su análisis y correlación.

Figura 17

Lista de agentes Wazuh registrados mostrando los 4 nodos monitoreados con sus IPs y estado.



PRUEBA Y VALIDACIONES

METODOLOGÍA DE PRUEBAS

Estas pruebas fueron realizadas de forma controlada desde la máquina Kali Linux con su red 10.0.40.10, esta ubicada en la red de administración, simulando el comportamiento de un atacante externo o interno que intenta comprometer los servicios de la infraestructura, se ejecutaron en un entorno de laboratorio cerrado, sin afectar sistemas productivos reales, las herramientas utilizadas para esta práctica fueron Nmap y Hydra siendo útiles en las pruebas ofensivas, por otro lado Wazuh tuvo como función dentro su misma plataforma la validación de detección.

RECONOCIMIENTO – NMAP

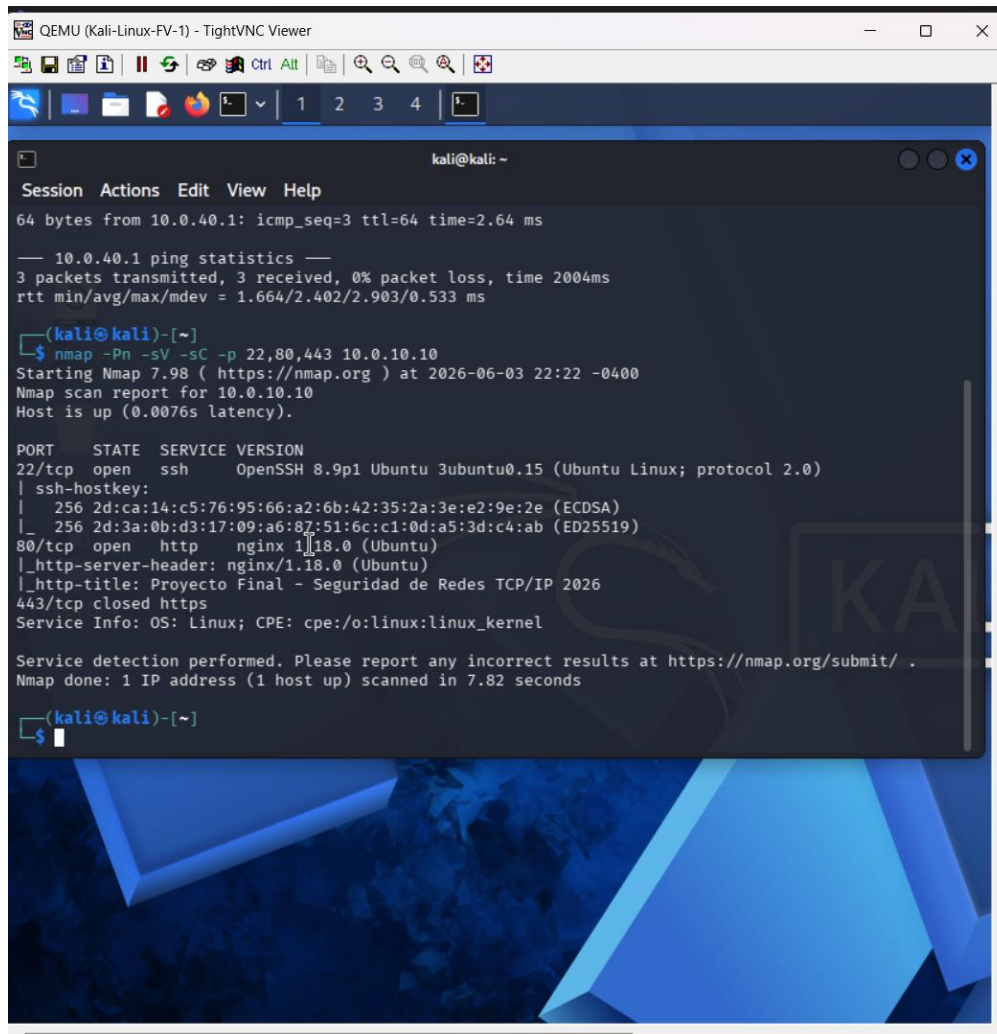
Como primera fase tenemos el reconocimiento de cualquier ataque, se hizo uso de la herramienta Nmap 7.98 para descubrir servidores activos, versiones de software y sistemas operativos en cada objetivo, para esto se empleó el parámetro -Pn para forzar el escaneo, aunque ICMP estuviera bloqueado por el firewall, lo cual refleja un escenario realista donde el atacante no puede hacer ping a los objetivos.

RECONOCIMIENTO CONTRA NGINX-DMZ

El escaneo reveló dos puertos abiertos: el puerto 22 con OpenSSH 8.9p1 y el puerto 80 con Nginx 1.18.0, esta versión exacta del servidor web fue expuesta en la respuesta HTTP, proporcionando información valiosa a un atacante para identificar vulnerabilidades conocidas asociadas a esa versión específica, el puerto 433 apareció cerrado, confirmando la ausencia de HTTPS en el servidor web.

Figura 18

Puestos abiertos y versiones de servicios expuestas.



```
QEMU (Kali-Linux-FV-1) - TightVNC Viewer
kali@kali: ~
Session Actions Edit View Help
64 bytes from 10.0.40.1: icmp_seq=3 ttl=64 time=2.64 ms
— 10.0.40.1 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.664/2.402/2.903/0.533 ms
(kali@kali)-[~]
$ nmap -Pn -sV -sC -p 22,80,443 10.0.10.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-03 22:22 -0400
Nmap scan report for 10.0.10.10
Host is up (0.0076s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_  256 2d:ca:14:c5:76:95:66:a2:6b:42:35:2a:3e:e2:9e:2e (ECDSA)
|_  256 2d:3a:0b:d3:17:09:a6:87:51:6c:c1:0d:a5:3d:c4:ab (ED25519)
80/tcp    open  http     nginx/1.18.0 (Ubuntu)
|_ http-server-header: nginx/1.18.0 (Ubuntu)
|_ http-title: Proyecto Final - Seguridad de Redes TCP/IP 2026
443/tcp   closed https
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

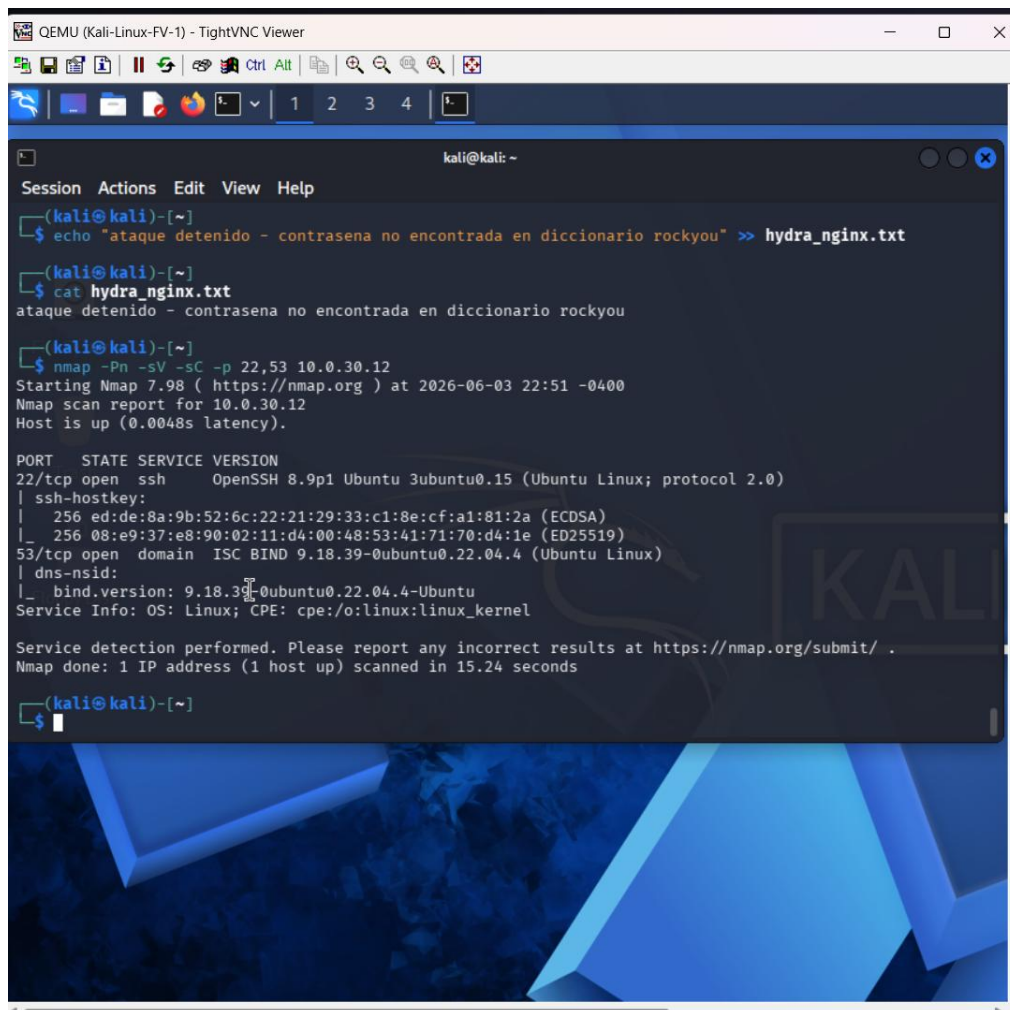
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.82 seconds
(kali@kali)-[~]
$
```

RECONOCIMIENTO CONTRA BLIND09-DNS

Se identificaron los puertos 22 con OpenSSH 8.9p1 y el puerto 53 con ISC BIND 9.18.39, un hallazgo crítico fue que la versión exacta de BIND fue expuesta mediante el registro bind.version, lo que permite a un atacante identificar CVEs específicos aplicables a esa versión.

Figura 19

Versión de BIND expuesta.

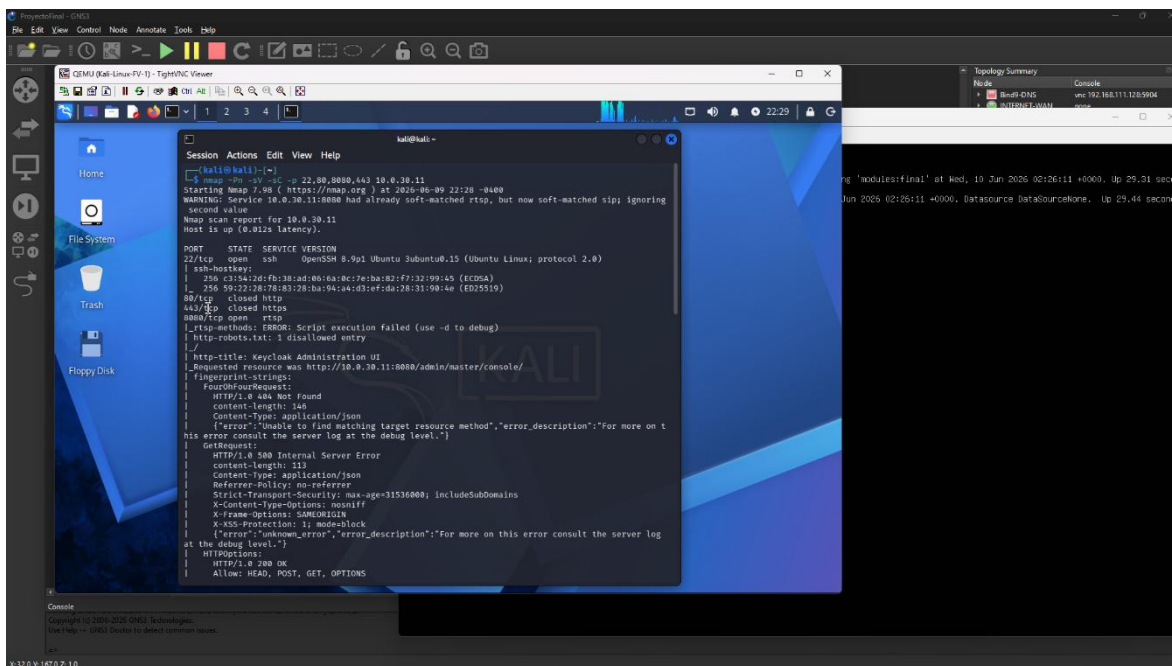


```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ echo "ataque detenido - contraseña no encontrada en diccionario rockyou" >> hydra_nginx.txt  
(kali@kali)-[~]  
$ cat hydra_nginx.txt  
ataque detenido - contraseña no encontrada en diccionario rockyou  
(kali@kali)-[~]  
$ nmap -Pn -sV -sC -p 22,53 10.0.30.12  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-03 22:51 -0400  
Nmap scan report for 10.0.30.12  
Host is up (0.0048s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)  
|_ ssh-hostkey:  
|_ 256 ed:de:8a:9b:52:6c:22:21:29:33:c1:8e:cf:a1:81:2a (ECDSA)  
|_ 256 08:e9:37:e8:90:02:11:d4:00:48:53:41:71:70:d4:1e (ED25519)  
53/tcp    open  domain   ISC BIND 9.18.39-0ubuntu0.22.04.4 (Ubuntu Linux)  
|_ dns-nsid:  
|_ bind.version: 9.18.39-0ubuntu0.22.04.4-Ubuntu  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 15.24 seconds  
(kali@kali)-[~]  
$
```

RECONOCIMIENTO CONTRA KEYCLOAK

El escaneo identificó el puerto 22 con OpenSSH y el puerto 8080 con el panel administración de Keycloak corriendo sobre HTTP sin cifrado, los puertos 80 y 443 aparecieron cerrados, confirmando que Keycloak no tiene HTTPS configurado, lo que implica que las credenciales de autenticación viajan en texto plano.

Figura 20
Keycloak mostrando el puerto 8080 abierto sin TLS.



ANÁLISIS DE VULNERABILIDADES WEB – NIKTO

Se hizo uso de Nikto 2.6.0 para realizar un análisis de vulnerabilidades sobre el servidor web Nginx en la DMZ, esto quiere decir que Nikto realizó 8,158 solicitudes HTTP en 35 segundos, identificando 8 a hallazgos de seguridad relacionados con la configuración del servidor.

Hallazgos identificados:

Tabla 3*Hallazgos identificados.*

#	Hallazgo	Riesgo
1	nginx/1.18.0 desactualizado (actual: 1.24.0)	Medio
2	Header Content-Security-Policy ausente	Medio
3	Header Referrer-Policy ausente	Medio
4	Header X-Content-Type-Options ausente	Medio
5	Header Strict-Transport-Security ausente	Medio
6	Header Permissions-Policy ausente	Medio
7	Header X-Frame-Options deprecado	Bajo
8	X-Content-Type-Options no configurado	Medio

Figura 21*Nikto contra Nginx-DMZ mostrando los 8 hallazgos de seguridad.*

```

QEMU (Kali-Linux-FV-1) - TightVNC Viewer
kali@kali: ~
Session Actions Edit View Help
command 'nikto' from deb nikto
Try: sudo apt install <deb name>

(kali@kali)-[~]
$ nikto -h http://10.0.10.10
- Nikto v2.6.0

+ Target IP: 10.0.10.10
+ Target Hostname: 10.0.10.10
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-06-03 22:24:51 (GMT-4)

+ Server: nginx/1.18.0 (Ubuntu)
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [600575] nginx/1.18.0 appears to be outdated (current is at least 1.24.0).

```

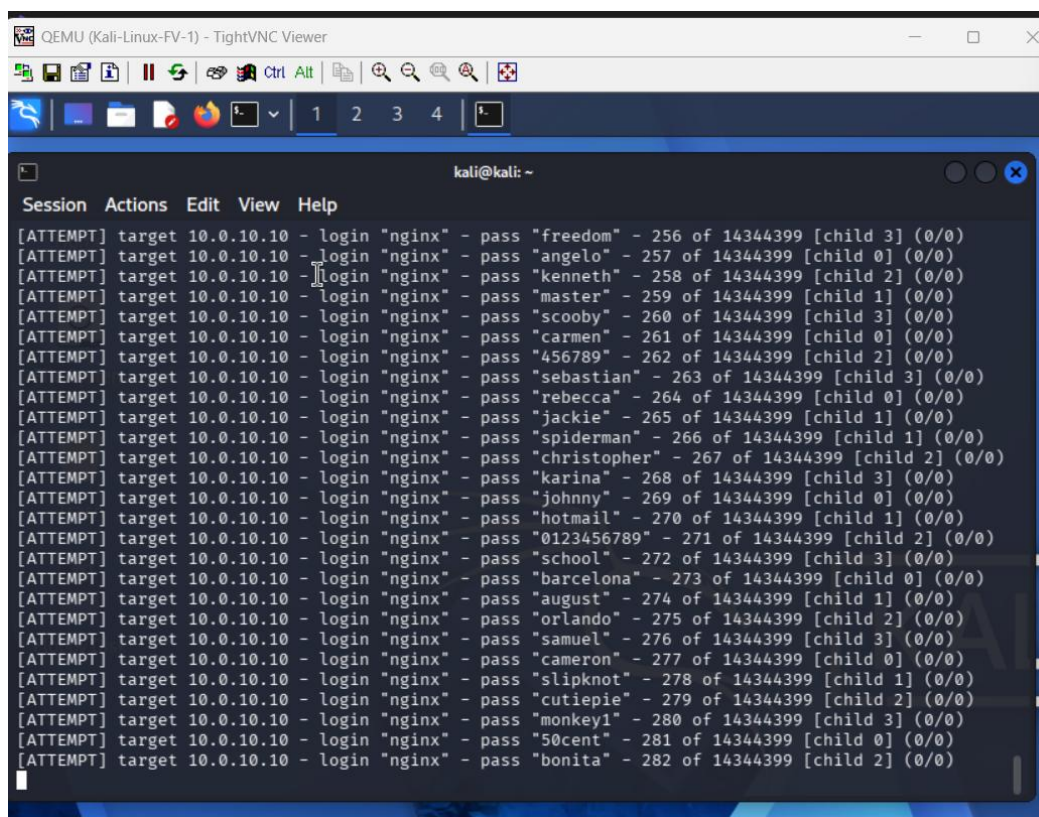

FUERZA BRUTA – HYDRA

FUERZA BRUTA SSH CONTRA NGINX-DMZ

Para este ataque se utilizó Hydra 9.6 con el fin de ejecutar un ataque de fuerza bruta controlado contra el servicio SSH del servidor Nginx, utilizando el diccionario Rockyou.txt que contiene 14,344,399 contraseñas comunes, este ataque fue configurado con 4 hilos paralelos para simular un escenario realista.

Figura 22

Hydra ejecutando ataque de fuerza bruta SSH contra Nginx-DMZ con intentos visibles.



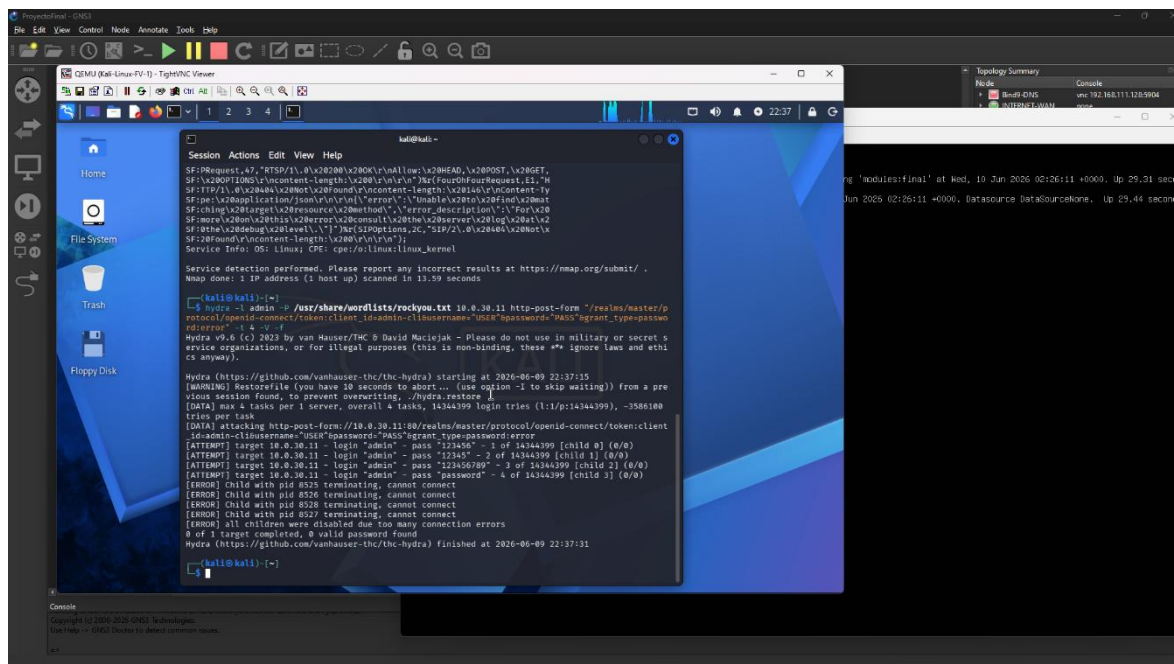
The screenshot shows a terminal window titled 'QEMU (Kali-Linux-FV-1) - TightVNC Viewer'. The terminal output displays a series of login attempts for the 'nginx' user on target '10.0.10.10'. Each attempt shows a password from the Rockyou.txt dictionary, the number of passwords tested (e.g., 256, 257, etc.), and the result '(0/0)' indicating failure. The attempts are distributed across four parallel child processes (child 0, 1, 2, 3). The passwords tested include: freedom, angelo, kenneth, master, scooby, carmen, 456789, sebastian, rebecca, jackie, spiderman, christopher, karina, johnny, hotmail, 0123456789, school, barcelona, august, orlando, samuel, cameron, slipknot, cutiepie, monkey1, 50cent, and bonita.

```
kali@kali: -
Session Actions Edit View Help
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "freedom" - 256 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "angelo" - 257 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "kenneth" - 258 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "master" - 259 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "scooby" - 260 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "carmen" - 261 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "456789" - 262 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "sebastian" - 263 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "rebecca" - 264 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "jackie" - 265 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "spiderman" - 266 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "christopher" - 267 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "karina" - 268 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "johnny" - 269 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "hotmail" - 270 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "0123456789" - 271 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "school" - 272 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "barcelona" - 273 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "august" - 274 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "orlando" - 275 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "samuel" - 276 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "cameron" - 277 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "slipknot" - 278 of 14344399 [child 1] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "cutiepie" - 279 of 14344399 [child 2] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "monkey1" - 280 of 14344399 [child 3] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "50cent" - 281 of 14344399 [child 0] (0/0)
[ATTEMPT] target 10.0.10.10 - login "nginx" - pass "bonita" - 282 of 14344399 [child 2] (0/0)
```

FUERZA BRUTA HTTP CONTRA KEYCLOAK

Se ejecutó un ataque de fuerza bruta contra el endpoint de autenticación de Keycloak, después de únicamente 4 intentos, Keycloak bloqueó automáticamente todas las conexiones de atacante mediante su mecanismo integrado de protección contra fuerza bruta, devolviendo el error all children were disabled due to many connection errors.

Figura 23
Hydra bloqueado por Keycloak tras 4 intentos.



ENUMERACIÓN DNS – ZONE TRANSFER ATTACK

Para las pruebas de enumeración DNS contra el servidor Bind9 se utilizaron herramientas dig y dnsrecon, esto con el fin de ejecutar una transferencia de zona AXFR, ya que una transferencia de zona exitosa mostraría todos los registros DNS internos de la organización, proporcionando al atacante un mapa completo de la infraestructura.

Resultado con dig AXFR:

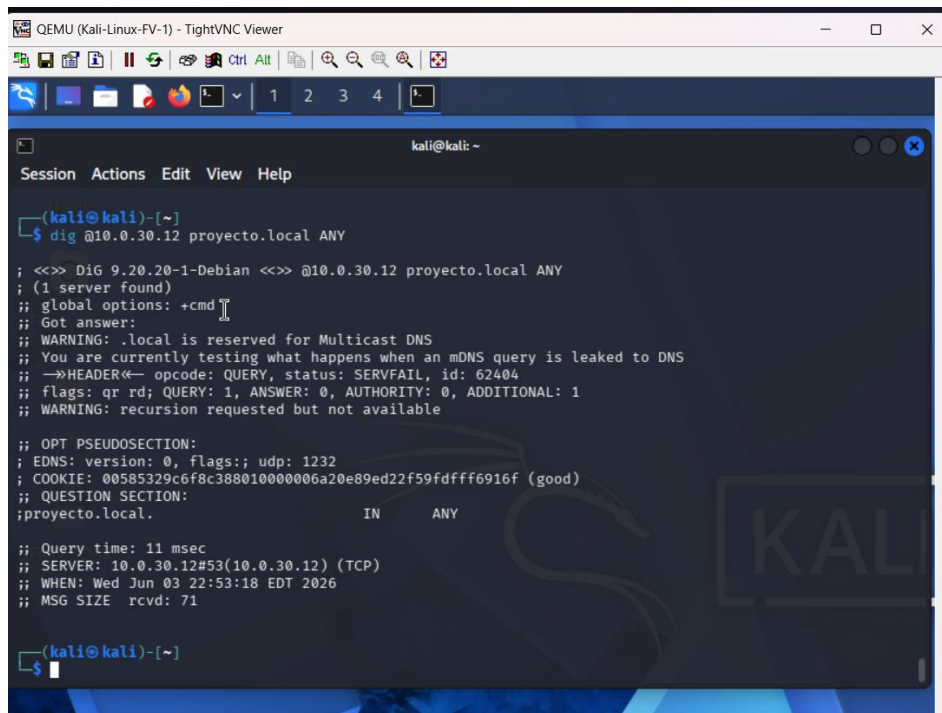
El servidor respondió con Transfer Failed, mostrando que las transferencias de zona están registradas.

Resultado con dnsrecon:

La herramienta confirmó ERROR axfr: No records were returned, dando como resultado un ataque bloqueado.

Figura 24

Resultado de dig AXFR mostrando Transfer failed contra BIND9-DNS.



```
QEMU (Kali-Linux-FV-1) - TightVNC Viewer
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ dig @10.0.30.12 proyecto.local ANY
; <<>> DiG 9.20.20-1-Debian <<>> @10.0.30.12 proyecto.local ANY
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
-->HEADER<-- opcode: QUERY, status: SERVFAIL, id: 62404
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

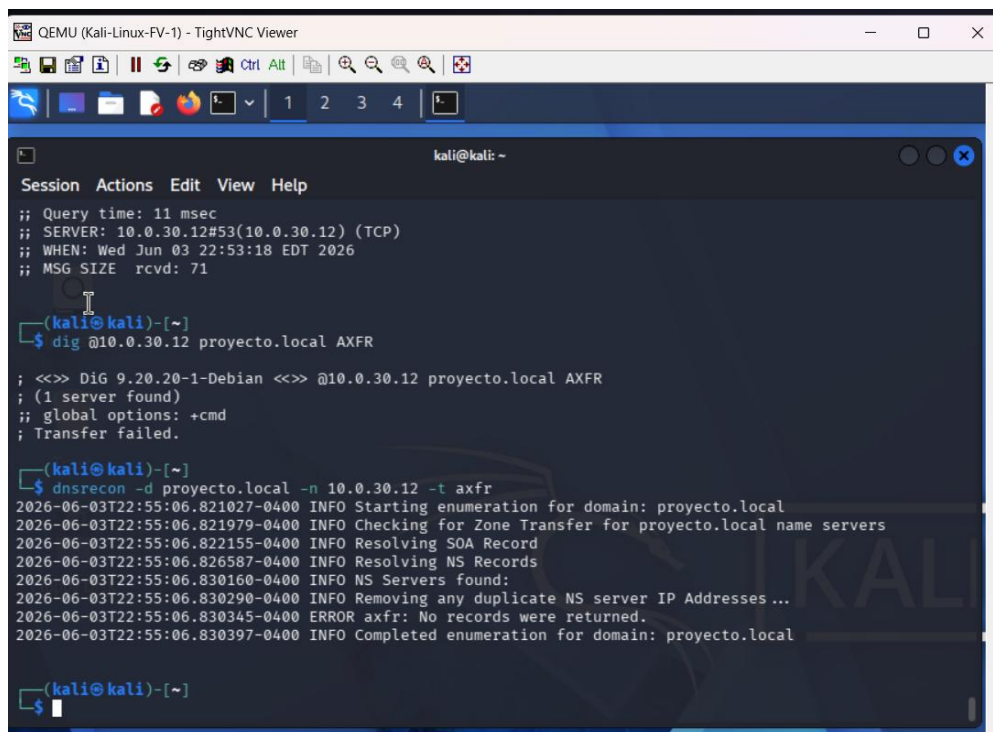
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 00585329c6f8c38801000006a20e89ed22f59fdfff6916f (good)
;; QUESTION SECTION:
;proyecto.local.                IN      ANY

;; Query time: 11 msec
;; SERVER: 10.0.30.12#53(10.0.30.12) (TCP)
;; WHEN: Wed Jun 03 22:53:18 EDT 2026
;; MSG SIZE rcvd: 71

(kali@kali)-[~]
$
```

Figura 25

Resultado de dnsrecon confirmando que la transferencia de zona fue denegada.



```
QEMU (Kali-Linux-FV-1) - TightVNC Viewer
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ dig @10.0.30.12 proyecto.local AXFR
; <<>> DiG 9.20.20-1-Debian <<>> @10.0.30.12 proyecto.local AXFR
; (1 server found)
;; global options: +cmd
; Transfer failed.

(kali@kali)-[~]
$ dnsrecon -d proyecto.local -n 10.0.30.12 -t axfr
2026-06-03T22:55:06.821027-0400 INFO Starting enumeration for domain: proyecto.local
2026-06-03T22:55:06.821979-0400 INFO Checking for Zone Transfer for proyecto.local name servers
2026-06-03T22:55:06.822155-0400 INFO Resolving SOA Record
2026-06-03T22:55:06.826587-0400 INFO Resolving NS Records
2026-06-03T22:55:06.830160-0400 INFO NS Servers found:
2026-06-03T22:55:06.830290-0400 INFO Removing any duplicate NS server IP Addresses...
2026-06-03T22:55:06.830345-0400 ERROR axfr: No records were returned.
2026-06-03T22:55:06.830397-0400 INFO Completed enumeration for domain: proyecto.local

(kali@kali)-[~]
$
```


VALIDACIÓN DE DETECCIÓN – WAZUH SIEM

Tras ejecutar todas las pruebas de ataque, se verificó en el dashboard de Wazuh que los eventos fueron detectados, correlacionados y alertados en tiempo real.

Resumen de detección:

Tabla 4

Detección Wazuh.

Ataque ejecutado	Detectado por Wazuh	Rule ID	Nivel
Fuerza bruta SSH	SI	5763	10
Múltiples intentos fallidos	SI	2502	10
Máximos intentos excedidos	SI	5758	8
Autenticación fallida	SI	5760	5

El total de alertas que generó Wazuh fueron 793 durante el periodo de pruebas, con 151 eventos de autenticación fallida correspondientes al ataque de Hydra.

Figura 26

Dashboard de Security Events en Wazuh mostrando 793 alertas con el pico de ataque visible en el gráfico.

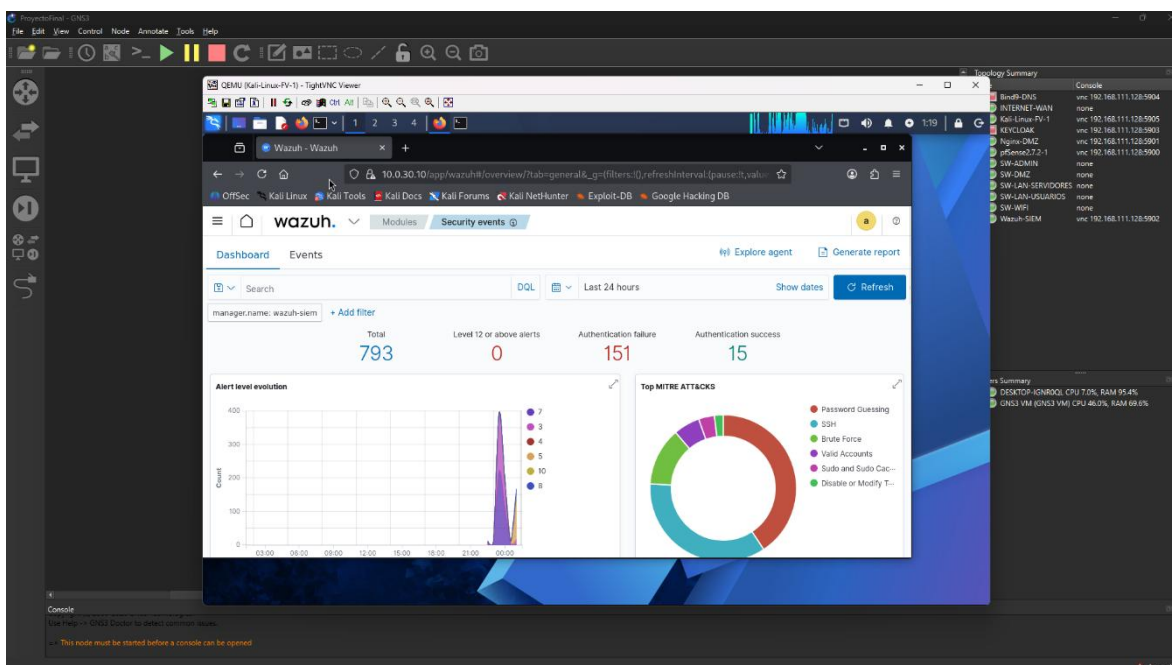


Figura 27

Lista de eventos de seguridad mostrando las alertas de fuerza bruta con rule ID 5763.

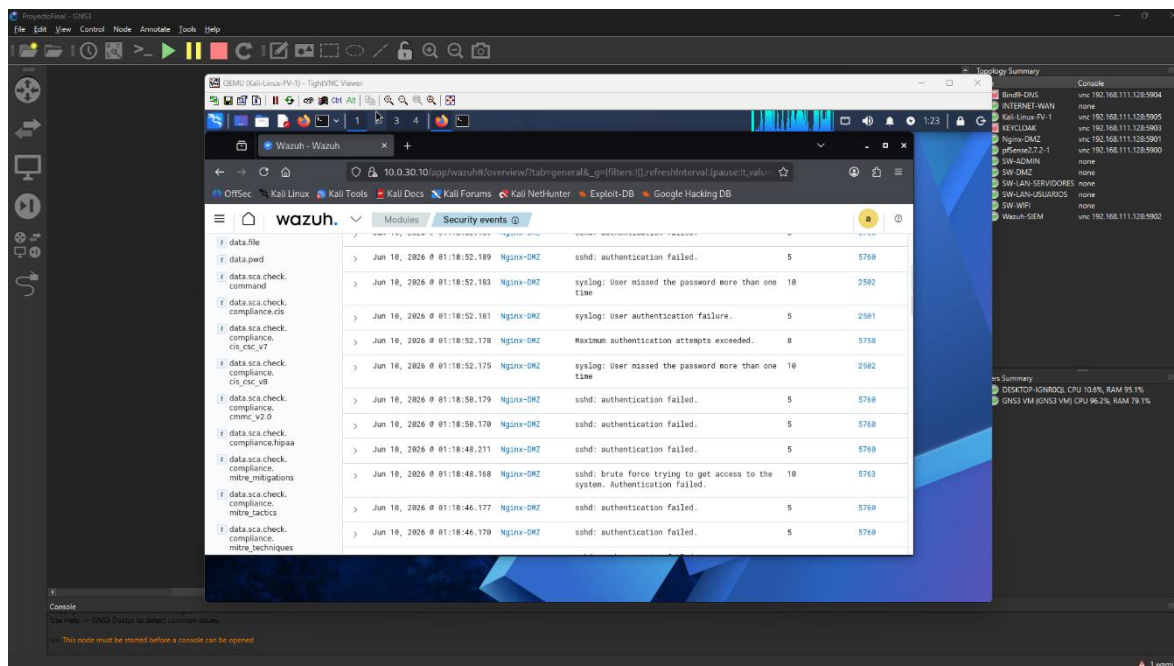


Figura 28

Detalle de la rule 5763 mostrando frequency 8, timeframe 120 y mapeo MITRE ATT&CK.

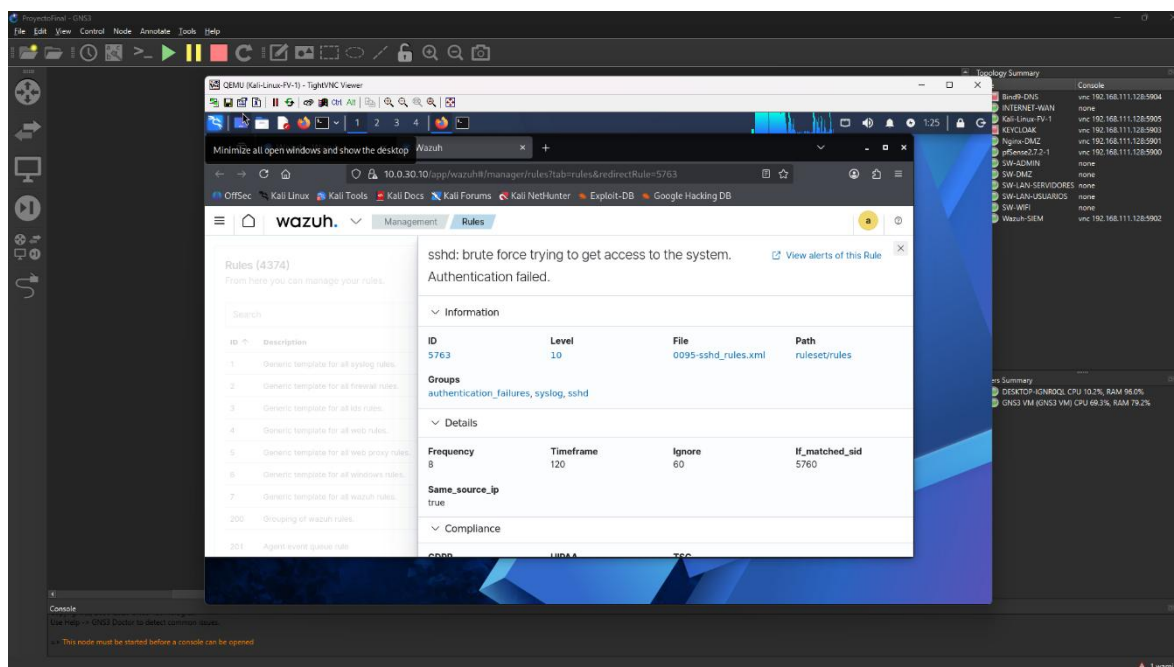
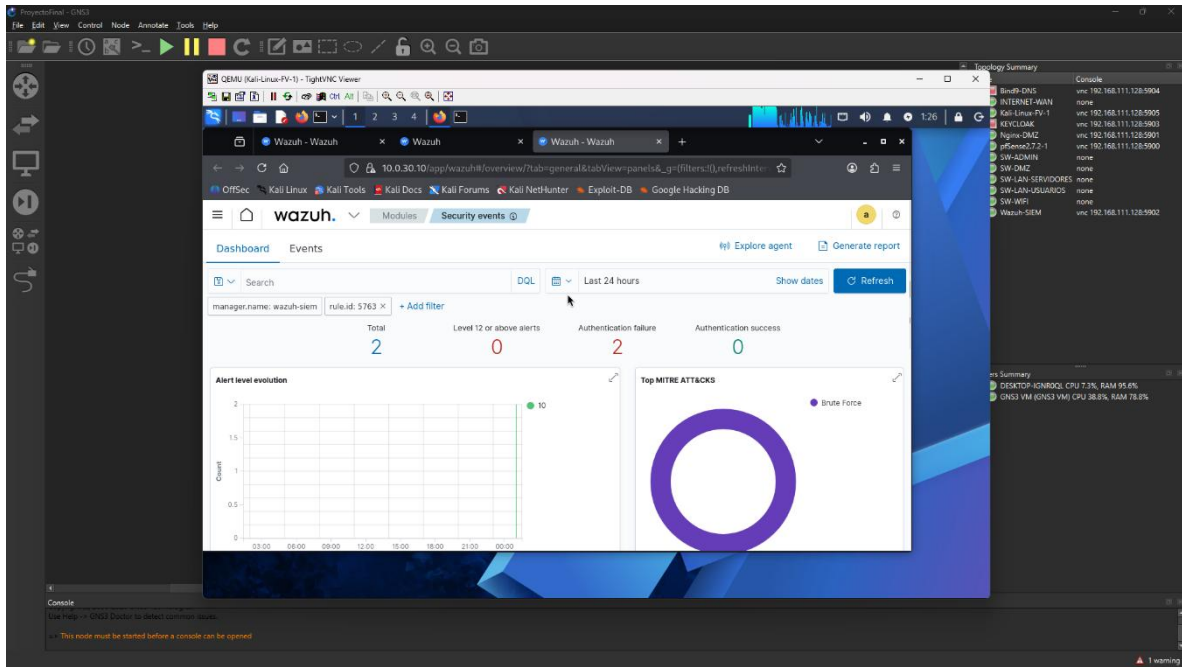


Figura 29

Dashboard filtrado por rule.id 5763 mostrando 2 alertas de Brute Force detectadas.



CONCLUSIONES

El presente proyecto demostró que es perfectamente viable construir un entorno de seguridad empresarial funcional, basado en el principio de defensa en profundidad, utilizando exclusivamente herramientas de código abierto integradas sobre una plataforma de simulación como GNS3. La infraestructura resultante, compuesta por un firewall segmentado, un SIEM, un gestor de identidades con MFA y una VPN, cumplió con los objetivos de diseño y permitió una validación práctica rigurosa.

La fase de pruebas controladas fue crucial para trascender la teoría y obtener una radiografía honesta de la postura de seguridad. Los hallazgos obtenidos revelan una realidad común en entornos reales: la coexistencia de controles fuertemente efectivos con debilidades de configuración significativas. Por un lado, se confirmó la robustez de controles como la segmentación de red, el bloqueo de transferencias de zona DNS, la detección de fuerza bruta por parte de Wazuh (con 793 alertas generadas y reglas personalizadas activadas) y la efectividad del MFA y el rate-limiting en Keycloak, que detuvo un ataque Hydra tras solo cuatro intentos.

Por otro lado, se identificaron riesgos latentes de severidad media y alta, como la exposición del panel de Keycloak sin cifrado TLS, la divulgación de versiones exactas de software (Nginx, BIND9) y la presencia de SSH en la zona DMZ, lo que amplifica la superficie de ataque. Estas debilidades no invalidan el diseño general, pero subrayan una lección fundamental: la detección y la alerta temprana (como la lograda con Wazuh) son indispensables, pero no sustituyen al endurecimiento preventivo. Una estrategia de seguridad madura debe combinar ambos enfoques en un ciclo de mejora continua.

Finalmente, se concluye que el valor de un proyecto de esta naturaleza no reside únicamente en el despliegue técnico, sino en la capacidad de documentar, analizar críticamente y extraer aprendizajes accionables. La trazabilidad desde el diseño hasta la evidencia de las pruebas ha proporcionado al equipo una comprensión profunda de las compensaciones (trade-offs) entre funcionalidad y seguridad, así como una hoja de ruta clara para futuras remediaciones, consolidando competencias esenciales para la gestión de ciberseguridad en entornos profesionales.

RECOMENDACIONES

Basado en los hallazgos documentados en la sección 10 del informe, se formulan las siguientes recomendaciones con el objetivo de mitigar los riesgos identificados y elevar el nivel de madurez de la infraestructura de seguridad:

1. Implementar cifrado TLS de forma prioritaria y generalizada: El hallazgo de severidad más alta es la exposición del panel de Keycloak en texto plano (HTTP, puerto 8080). Se recomienda configurar un certificado SSL/TLS válido (autofirmado para pruebas o de una CA reconocida en producción) para todo el tráfico hacia Keycloak, redirigiendo todo el tráfico del puerto 80 al 443 de forma forzosa. Adicionalmente, se debe habilitar HTTPS en el servidor web Nginx para proteger cualquier comunicación con la DMZ.
2. Reducir la superficie de ataque expuesta: Se debe restringir el acceso al servicio SSH en el servidor Nginx-DMZ. La recomendación es implementar una regla en pfSense que permita conexiones SSH únicamente desde la dirección IP de la red de administración (10.0.40.0/24). Si es estrictamente necesario el acceso desde internet, se debería desplegar un bastion host o, idealmente, requerir la conexión a la VPN OpenVPN primero.
3. Endurecer la configuración de los servicios expuestos:
 - Ocultar versiones: Modificar los archivos de configuración de Nginx (server_tokens off;) y BIND9 (version "none";) para que no revelen la versión exacta del software en las respuestas a clientes externos.
 - Mejorar headers HTTP: Configurar en Nginx los headers de seguridad recomendados por Nikto, incluyendo Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, y Referrer-Policy para mitigar ataques del lado del cliente.
 - Actualizar componentes: Establecer un plan de actualización de software para Nginx (de la versión 1.18.0 a la 1.24.x o superior) a fin de corregir vulnerabilidades conocidas.
4. Fortalecer la autenticación en servicios de administración: Dado que el ataque de fuerza bruta contra SSH fue detectable pero exitoso en ejecutarse, se recomienda, además del punto 2, deshabilitar la autenticación por contraseña para SSH en todos los servidores y

forzar el uso de pares de llaves criptográficas (autenticación asimétrica) para el acceso administrativo.

5. Mejorar la respuesta ante incidentes: Aunque Wazuh detectó el ataque de fuerza bruta, no se evidenció una respuesta automática. Se recomienda configurar Wazuh (o pfSense) para que, tras un número determinado de alertas (ej. regla 5763), ejecute una acción de respuesta, como bloquear automáticamente la dirección IP ofensiva mediante una regla dinámica en el firewall, convirtiendo la detección en una prevención activa.
6. Ampliar el alcance de las pruebas de seguridad: Para cerrar la brecha entre diseño y validación, se recomienda ejecutar un ciclo de pruebas específico para el segmento de red WiFi (10.0.50.0/24) evaluando la robustez de su cifrado, la segmentación efectiva desde esa zona hacia otras redes, y la capacidad de detección de dispositivos maliciosos conectados a ella.

REFERENCIAS BIBLIOGRAFICAS

- GNS3 Project. (2024). *GNS3: The software that empowers network professionals*. Recuperado de <https://www.gns3.com/>
- Netgate. (2023). *pfSense® Software Documentation*. Recuperado de <https://docs.netgate.com/pfsense/en/latest/>
- OISF (Open Information Security Foundation). (2024). *Suricata User Guide*. Recuperado de <https://suricata.readthedocs.io/>
- The Zeek Project. (2024). *Zeek User Manual*. Recuperado de <https://docs.zeek.org/>
- Wazuh, Inc. (2024). *Wazuh Documentation: Open Source XDR. Open Source SIEM*. Recuperado de <https://documentation.wazuh.com/>
- Keycloak Project. (2024). *Keycloak Documentation*. Recuperado de <https://www.keycloak.org/documentation>
- F5, Inc. (2024). *NGINX Documentation*. Recuperado de <https://nginx.org/en/docs/>
- Internet Systems Consortium. (2023). *BIND 9 Administrator Reference Manual*. Recuperado de <https://bind9.readthedocs.io/>
- OpenVPN, Inc. (2024). *OpenVPN Community Resources*. Recuperado de <https://openvpn.net/community-resources/>
- Lyon, G. F. (2009). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. [Insecure.Com](https://www.insecure.com), LLC.
- Sullo, C. (2024). *Nikto2: Web Server Scanner*. Recuperado de <https://github.com/sullo/nikto>
- Van Hauser, T. (2024). *THC-Hydra: A very fast network logon cracker*. Recuperado de <https://github.com/vanhauser-thc/thc-hydra>

LINK DE VIDEOS

https://drive.google.com/drive/folders/1Fr0nXzUjHg38lqkyW99IWT_ZKlwRMw7E?usp=sharing